MSfinal

Yael Scharf

2022-06-10

Introduction

Multiple sclerosis (MS) is an autoimmune condition that can affect the brain and spinal cord, causing a wide range of symptoms, such as problems with vision and arm or leg movement, sensation, or balance. MS is a lifelong condition, mainly diagnosed in people in their 20s, 30s, and 40s, and is 2-3 times more common in women than in men. Although MS is rarely fatal, the average life expectancy for people with MS is around 5 to 10 years lower than average, and this gap appears to be getting smaller all the time. Biologically, in MS, the immune system mistakenly attacks the healthy parts of the body: the brain or spinal cord of the nervous system, mainly the layer that surrounds and protects the nerves called the myelin sheath. Hence, messages traveling along the nerves become slower or disrupted. The causes of the behavior described above are unclear, but most experts think a combination of genetic and environmental factors is involved. There's currently no cure for MS, but several treatments can help control the condition and ease symptoms to improve the quality of life of people with this condition. (For more information on MS: https://www.nhs.uk/conditions/multiple-sclerosis/) Most of the studies of MS center on the immune system, but Fitzgerald et al. 2021 decided to take a different approach by studying the metabolome of people with MS. In this report, we will analyze this unique research's data set with the aim of finding new correlations using four classifier models: K-Nearest Neighbors (KNN), K-means, random forest and Support Vector Machine (SVM); and the feature reduction model T-distributed Stochastic Neighbor Embedding (t-SNE). This report will be based on the Fitzgerald et al. 2021 data set (supplementary information):

```
# Reading the data
ms_dat <- read.delim("/Users/yaelscharf/Downloads/metab_data_template_final.csv", header = TRUE, sep=",
str(ms_dat)
```

```
## 'data.frame':    954 obs. of  275 variables:
##  $ record_id: int  1 2 3 3 4 4 5 5 6 6 ...
##  $ status   : chr  "MS" "MS" "HC" "HC" ...
##  $ age      : int  37 36 39 40 66 67 52 54 52 53 ...
##  $ gender   : chr  "F" "F" "F" "F" ...
##  $ race     : chr  "Black or African American" "White" "White" "White" ...
##  $ batch    : int  7 7 5 5 5 5 5 5 5 5 ...
##  $ X48762   : num  14 13.2 15.1 16.1 12.7 ...
##  $ X20675   : num  18.7 18.6 18.4 18.3 18 ...
##  $ X34400   : num  14.3 14.7 14.9 14.5 14.6 ...
##  $ X30460   : num  12.3 13.2 12.5 13.3 12.7 ...
##  $ X32350   : num  13.9 14.2 14.1 14.3 14.7 ...
##  $ X34395   : num  12.7 12.7 15 14.4 13.5 ...
##  $ X34389   : num  13.4 14.2 15.9 15.1 14.6 ...
##  $ X38395   : num  12.5 12.5 14.2 13.5 12.9 ...
##  $ X38168   : num  15.6 14.9 14.7 14.2 14.9 ...
##  $ X38276   : num  12.7 14.2 15.6 15.5 13.9 ...
```

```
##  $ X42374   : num  17.7 17.9 17.7 17.8 17.7 ...
##  $ X43761   : num  14.6 13.5 14 14 14.3 ...
##  $ X43343   : num  12.3 12.2 12.6 12.2 12 ...
##  $ X43266   : num  13.6 12 14.1 13.5 14.1 ...
##  $ X36746   : num  14 13.9 13.7 13.4 13.5 ...
##  $ X42489   : num  14.6 14.5 15.5 14.8 14.3 ...
##  $ X18281   : num  12.7 16 15.9 16.5 14.8 ...
##  $ X22036   : num  13.3 13.4 13.9 13.4 12.9 ...
##  $ X35675   : num  17.5 18 17.8 17.6 17.8 ...
##  $ X45095   : num  14.7 14.1 14 14.2 14.1 ...
##  $ X46115   : num  13.4 13.4 13.9 13.5 14.1 ...
##  $ X32197   : num  16.2 15.8 16.2 16.3 16.1 ...
##  $ X31787   : num  14.9 17.5 16.9 16.8 14.8 ...
##  $ X32397   : num  13.7 14.4 14 14.1 14.1 ...
##  $ X531     : num  15.7 15.7 15.9 15.8 16.2 ...
##  $ X542     : num  15.8 18.8 16 15.9 19.2 ...
##  $ X22053   : num  13.9 14.5 13.7 14 14.1 ...
##  $ X39600   : num  15.9 13.6 16.3 15.3 14.9 ...
##  $ X1549    : num  15.8 16.1 16 15.8 16.5 ...
##  $ X32457   : num  13.2 14.5 13.4 13.5 14.3 ...
##  $ X22001   : num  14 14.3 13.8 14.3 14 ...
##  $ X48448   : num  12.9 14.6 16.8 17.3 14.9 ...
##  $ X27672   : num  17.8 16.4 17.4 17.6 17.5 ...
##  $ X48763   : num  13 14.1 14.7 15.4 12.8 ...
##  $ X12017   : num  13.4 13.7 13.9 13.1 13.9 ...
##  $ X46165   : num  12.5 14.4 15.8 16.2 14.1 ...
##  $ X46548   : num  13.6 13.5 14.6 14.4 13.7 ...
##  $ X15677   : num  15.3 16.6 14.2 17.1 14.7 ...
##  $ X32445   : num  14.2 15 15.8 15.9 13.5 ...
##  $ X15749   : num  13 13.3 15.6 15.1 12.9 ...
##  $ X1558    : num  14.2 14.7 14.4 14.8 14.7 ...
##  $ X37181   : num  13.2 11.6 14.5 13.5 13.1 ...
##  $ X36099   : num  13.2 12.8 14.1 14.3 14.6 ...
##  $ X48441   : num  15.1 15.9 15.2 15.4 16.1 ...
##  $ X35527   : num  14.3 13.4 13.2 13.6 14.5 ...
##  $ X1669    : num  13.7 13 13 13.6 13.5 ...
##  $ X46146   : num  15.5 15.3 16.1 17.3 17.3 ...
##  $ X36098   : num  15.7 13.8 16.3 16.7 16 ...
##  $ X1418    : num  13.4 13.2 13 13.1 13.3 ...
##  $ X34424   : num  15.3 16.1 16.9 16.6 15.5 ...
##  $ X35136   : num  13.6 13.3 14 14.1 14 ...
##  $ X1494    : num  17.3 17.4 17.7 17.5 17.6 ...
##  $ X37186   : num  10.9 13 12.3 12.8 12.8 ...
##  $ X37190   : num  12.4 15.9 14.6 14.5 14.8 ...
##  $ X37198   : num  14.9 14.6 16.4 14.5 14.9 ...
##  $ X37200   : num  12.6 12.5 16.1 13.8 13.9 ...
##  $ X37196   : num  12.2 12.1 15.7 12.7 13.6 ...
##  $ X43231   : num  12.8 13 13.3 13.4 13 ...
##  $ X36776   : num  15.1 15.2 14.7 14.7 15.2 ...
##  $ X35114   : num  15.1 15.3 15.2 15.1 15.3 ...
##  $ X34390   : num  14 13.8 15.1 15.5 13.4 ...
##  $ X32198   : num  19 19.6 19.1 18.9 19.7 ...
##  $ X43258   : num  12.5 11.4 12.5 12.2 13 ...
##  $ X1126    : num  20 19.8 20 20.3 20.1 ...
```

```
##  $ X22132   : num  13.6 13.9 13.4 13.4 13.5 ...
##  $ X32827   : num  12.3 12.5 12.3 12 12.4 ...
##  $ X37209   : num  12.8 14.6 14.2 14.3 14.9 ...
##  $ X37202   : num  16.4 17.9 16.8 16.7 16.4 ...
##  $ X37203   : num  15.7 15.8 15.4 15.1 15.7 ...
##  $ X37211   : num  14.4 14.9 14.6 15 14 ...
##  $ X37210   : num  13.1 13.5 12.8 12.7 11.7 ...
##  $ X31591   : num  14.6 17.1 16.4 16.4 16.8 ...
##  $ X1638    : num  20.4 20.1 19.8 20.4 20.1 ...
##  $ X512     : num  19.2 19 19.3 19.2 19.1 ...
##  $ X12129   : num  14.5 14.8 14.6 14.6 14.1 ...
##  $ X3141    : num  20.7 20 19.9 20.1 20.2 ...
##  $ X32586   : num  15.6 16.3 15.4 15.5 16.3 ...
##  $ X43807   : num  17.3 17.5 17.7 17.5 17.6 ...
##  $ X2137    : num  13.8 14.4 14.3 14.4 14.6 ...
##  $ X32412   : num  15.1 15.4 14.9 14.9 15.6 ...
##  $ X48782   : num  12.8 13 13 13.1 13.2 ...
##  $ X569     : num  17.2 18.6 18.5 18.8 17.6 ...
##  $ X1642    : num  16.4 17.2 16.5 17.8 16.8 ...
##  $ X32492   : num  15.7 15.5 15.8 17 15.7 ...
##  $ X15500   : num  20.8 20.5 20.6 20.7 20.4 ...
##  $ X35320   : num  16 15.2 17.6 18.2 16.4 ...
##  $ X1563    : num  16 16.1 15.8 15.7 16.4 ...
##  $ X22842   : num  13.2 13 13.9 13.3 15 ...
##  $ X15506   : num  19.2 18.9 18.9 18.6 19 ...
##  $ X34396   : num  15 16 15.9 15.8 15.5 ...
##  $ X38637   : num  12.4 11.5 15 14.4 13.9 ...
##  $ X38178   : num  16.1 16.5 15.9 15.1 16.7 ...
##  $ X1564    : num  18.9 19.1 19.2 19.1 19.5 ...
##   [list output truncated]
```

Since the data is clean from NA's and missing values, we can look closer at the data. The data shows that the people that participated in the research are mainly categorized according to status: Multiple Sclerosis (MS) and Healthy Control (HC), gender and race. Furthermore, each metabolite is marked with an X and a serial number. For further analysis, we'll need more information on the distribution of these categories. Hence, we'll turn these categories to factors.

```
cols <- c("status", "gender", "race")
ms_dat[cols] <- lapply(ms_dat[cols], factor)
summary(ms_dat[cols])
```

```
##  status   gender                          race
##  HC:317   F:692                             : 40
##  MS:637   M:262   Black or African American: 92
##                   Other                     : 27
##                   White                     :795
```

The summary above indicates that approximately 2/3 of the participants are with MS; 70% of them are female and 80% of white ethnicity. This leads to the question whether there is any connection between these different factors to the levels of metabolites, and if so, how. In order to answer this question, we need to check how each of the factors correlate with the data.

EDA

Note: In lecture the images had mistakes because of errors only much later found in the code. In this section are the correct results. By the order of the data, we'll start with status of health(MS or HC).

```
# Random forest requires setting a seed
set.seed(123)
ms_split <- initial_split(ms_dat %>% dplyr::select(-c(record_id, batch)),
                          strata = status)
ms_train_stat <- training(ms_split)
ms_test_stat  <- testing(ms_split)

# Checking the results of the split
nrow(ms_train_stat)
```

```
## [1] 714
```

```
nrow(ms_train_stat)/nrow(ms_split)
```

```
## analysis
##        1
```

```
# Training set proportions by status of health
ms_train_stat %>%
  count(status) %>%
  mutate(prop = n/sum(n))
```

```
##   status   n      prop
## 1     HC 237 0.3319328
## 2     MS 477 0.6680672
```

```
# Testing set proportions by status of health
ms_test_stat %>%
  count(status) %>%
  mutate(prop = n/sum(n))
```

```
##   status   n      prop
## 1     HC  80 0.3333333
## 2     MS 160 0.6666667
```

Seeing that the proportions of the training and test sets are more or less similar to the data set, the next step will be to define the number of trees in the random forest model, 1000, and the engine "ranger":

```
# Creating random forest model
rf_mod_stat <-
  rand_forest(trees = 1000) %>%
  set_engine("ranger") %>%
  set_mode("classification")
```

With a defined model, we can start the training step with new seed for reproducible results:

```r
# Training the model
set.seed(345)
rf_fit_stat <-
  rf_mod_stat %>%
  fit(status ~ ., data = ms_train_stat)
rf_fit_stat
```

```
## parsnip model object
##
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, num.trees = ~1000,      num.threads = 1, verbose = F
##
## Type:                             Probability estimation
## Number of trees:                  1000
## Sample size:                      714
## Number of independent variables:  272
## Mtry:                             16
## Target node size:                 10
## Variable importance mode:         none
## Splitrule:                        gini
## OOB prediction error (Brier s.):  0.1793534
```

It seems that the requirements and execution of our model were met with a low rate of prediction error, 0.18. Hence, we'll test the trade-off between the sensitivity and specificity and the classification accuracy using the Receiver Operating Characteristic (ROC).

```r
# Calculating our model's prediction abilities
rf_training_stat_pred <-
  predict(rf_fit_stat, ms_train_stat) %>%
  bind_cols(predict(rf_fit_stat, ms_train_stat, type = "prob")) %>%
  bind_cols(ms_train_stat %>% dplyr::select(status))

# ROC calculation
rf_training_stat_pred %>%
  roc_auc(truth = status, .pred_MS)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary             0
```

```r
# Accuracy calculation
rf_training_stat_pred %>%
  accuracy(truth = status, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary             1
```

Although the accuracy is high (1!), the ROC estimate is quite low (0!). Let's check what happens when we perform the same analysis on the test data:

```
rf_testing_stat_pred <-
  predict(rf_fit_stat, ms_test_stat) %>%
  bind_cols(predict(rf_fit_stat, ms_test_stat, type = "prob")) %>%
  bind_cols(ms_test_stat %>% dplyr::select(status))

# ROC calculation
rf_testing_stat_pred %>%
  roc_auc(truth = status, .pred_MS)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.286
```

```
# Accuracy calculation
rf_testing_stat_pred %>%
  accuracy(truth = status, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.717
```

These results of the test set seem more realistic. The ROC is still quite low (0.28), but the accuracy is more realistic than the value of 1 (0.72). However, both of the values indicate that the model is not good yet. Hence, using 10-fold cross validation re-sampling we'll try to improve it:

```
# Creating 10 mini data sets from the training data
set.seed(456)
folds <- vfold_cv(ms_train_stat, v = 10)
folds
```

```
## #  10-fold cross-validation
## # A tibble: 10 x 2
##    splits           id
##    <list>           <chr>
##  1 <split [642/72]> Fold01
##  2 <split [642/72]> Fold02
##  3 <split [642/72]> Fold03
##  4 <split [642/72]> Fold04
##  5 <split [643/71]> Fold05
##  6 <split [643/71]> Fold06
##  7 <split [643/71]> Fold07
##  8 <split [643/71]> Fold08
##  9 <split [643/71]> Fold09
## 10 <split [643/71]> Fold10
```

```
# Setting the random forest workflow that bundles together the model specifications and actual modeling
rf_stat_wf <-
  workflow() %>%
  add_model(rf_mod_stat) %>%
  add_formula(status ~ .)

# Add folds to workflow and train model
set.seed(567)
rf_fit_stat_rs <-
  rf_stat_wf %>%
  fit_resamples(folds)

rf_fit_stat_rs
```

```
## # Resampling results
## # 10-fold cross-validation
## # A tibble: 10 x 4
##    splits           id     .metrics         .notes
##    <list>           <chr>  <list>           <list>
##  1 <split [642/72]> Fold01 <tibble [2 x 4]> <tibble [0 x 3]>
##  2 <split [642/72]> Fold02 <tibble [2 x 4]> <tibble [0 x 3]>
##  3 <split [642/72]> Fold03 <tibble [2 x 4]> <tibble [0 x 3]>
##  4 <split [642/72]> Fold04 <tibble [2 x 4]> <tibble [0 x 3]>
##  5 <split [643/71]> Fold05 <tibble [2 x 4]> <tibble [0 x 3]>
##  6 <split [643/71]> Fold06 <tibble [2 x 4]> <tibble [0 x 3]>
##  7 <split [643/71]> Fold07 <tibble [2 x 4]> <tibble [0 x 3]>
##  8 <split [643/71]> Fold08 <tibble [2 x 4]> <tibble [0 x 3]>
##  9 <split [643/71]> Fold09 <tibble [2 x 4]> <tibble [0 x 3]>
## 10 <split [643/71]> Fold10 <tibble [2 x 4]> <tibble [0 x 3]>
```

```
collect_metrics(rf_fit_stat_rs)
```

```
## # A tibble: 2 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy binary     0.738    10  0.0107 Preprocessor1_Model1
## 2 roc_auc  binary     0.790    10  0.0116 Preprocessor1_Model1
```

The re-sampling improved the ROC estimate significantly (0.79), while still maintaining the low value previously observed in the testing of the accuracy estimates is maintained (0.73). To understand which of all the estimates previously observed reflects our data set the most, we need to make predictions of these estimates:

```
# ROC calculation
rf_testing_stat_pred %>%
  roc_auc(truth = status, .pred_MS)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.286
```

```
# Accuracy calculation
rf_testing_stat_pred %>%
  accuracy(truth = status, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.717
```

The estimates remain unchanged from the data set, indicating that the current model still does not classify well (accuracy 0f ~0.7 and ROC of 0.28). Tuning may solve that by using one decision tree that is more sensitive to parameters that random forest isn't.

```
# Defining the parameters for the decision tree
tune_stat <-
  decision_tree(
    cost_complexity = tune(),
    tree_depth = tune()
  ) %>%
  set_engine("rpart") %>%
  set_mode("classification")

tune_stat
```

```
## Decision Tree Model Specification (classification)
##
## Main Arguments:
##   cost_complexity = tune()
##   tree_depth = tune()
##
## Computational engine: rpart
```

```
# Creating 5 'datasets' with different combinations
tree_grid <- grid_regular(cost_complexity(),
                          tree_depth(),
                          levels = 5)

tree_grid
```

```
## # A tibble: 25 x 2
##    cost_complexity tree_depth
##              <dbl>      <int>
##  1    0.0000000001          1
##  2    0.0000000178          1
##  3    0.00000316            1
##  4    0.000562              1
##  5    0.1                   1
##  6    0.0000000001          4
##  7    0.0000000178          4
##  8    0.00000316            4
##  9    0.000562              4
## 10    0.1                   4
## # ... with 15 more rows
```

```
tree_grid %>%
  count(tree_depth)
```

```
## # A tibble: 5 x 2
##   tree_depth     n
##        <int> <int>
## 1          1     5
## 2          4     5
## 3          8     5
## 4         11     5
## 5         15     5
```

```
set.seed(678)
ms_folds_stat <- vfold_cv(ms_train_stat) # create the actual cross-validation folds
```

Now let's add the grid to this model:

```
set.seed(789)
# Defining workflow
tree_stat_wf <-workflow() %>%
  add_model(tune_stat) %>%
  add_formula(status ~ .)

# Updating and training workflow with grids
tree_stat_res <-
  tree_stat_wf %>%
  tune_grid(
    resamples = ms_folds_stat,
    grid = tree_grid
    )

tree_stat_res
```

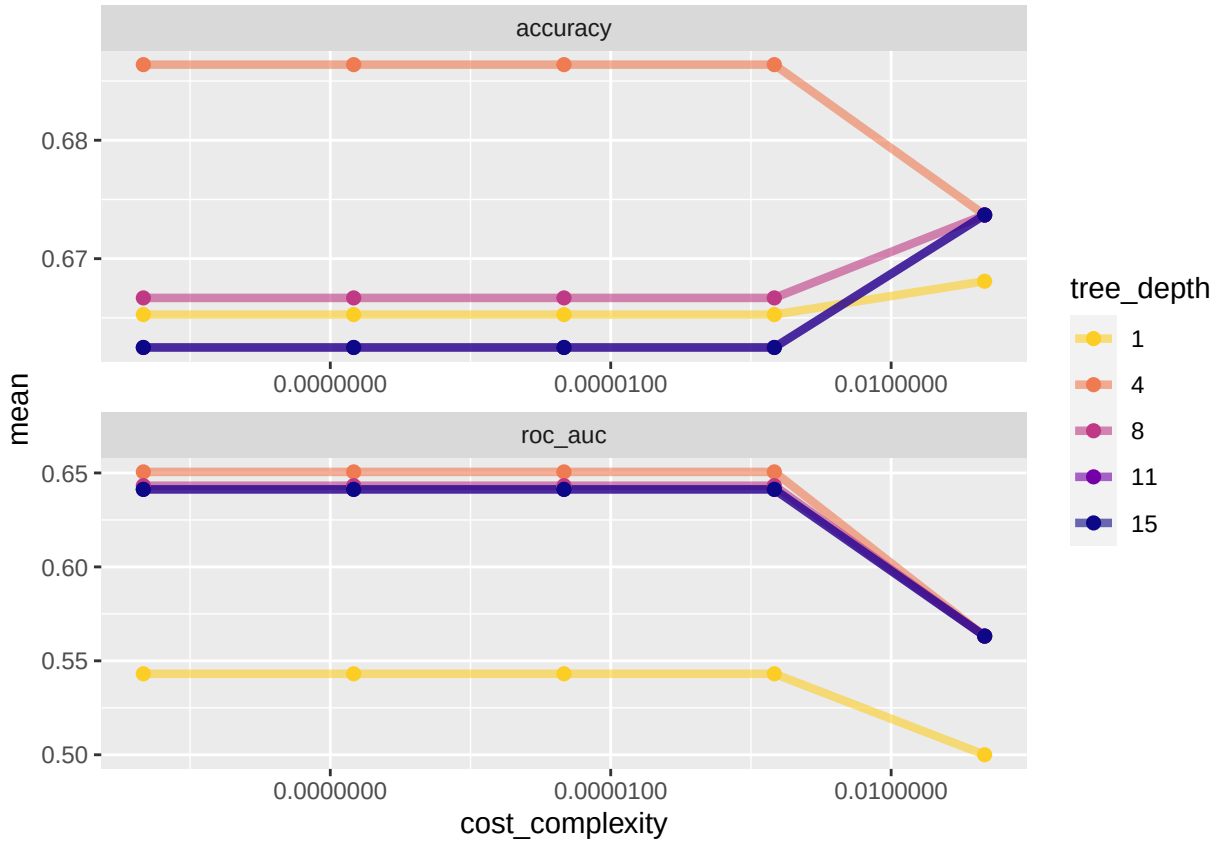
```
## # Tuning results
## # 10-fold cross-validation
## # A tibble: 10 x 4
##    splits           id     .metrics          .notes
##    <list>           <chr>  <list>            <list>
##  1 <split [642/72]> Fold01 <tibble [50 x 6]> <tibble [0 x 3]>
##  2 <split [642/72]> Fold02 <tibble [50 x 6]> <tibble [0 x 3]>
##  3 <split [642/72]> Fold03 <tibble [50 x 6]> <tibble [0 x 3]>
##  4 <split [642/72]> Fold04 <tibble [50 x 6]> <tibble [0 x 3]>
##  5 <split [643/71]> Fold05 <tibble [50 x 6]> <tibble [0 x 3]>
##  6 <split [643/71]> Fold06 <tibble [50 x 6]> <tibble [0 x 3]>
##  7 <split [643/71]> Fold07 <tibble [50 x 6]> <tibble [0 x 3]>
##  8 <split [643/71]> Fold08 <tibble [50 x 6]> <tibble [0 x 3]>
##  9 <split [643/71]> Fold09 <tibble [50 x 6]> <tibble [0 x 3]>
## 10 <split [643/71]> Fold10 <tibble [50 x 6]> <tibble [0 x 3]>
```

```
tree_stat_res %>%
  collect_metrics()
```

```
## # A tibble: 50 x 8
##    cost_complexity tree_depth .metric  .estimator  mean     n std_err .config
##              <dbl>      <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
##  1    0.0000000001          1 accuracy binary     0.665    10 0.00816 Preproces~
##  2    0.0000000001          1 roc_auc  binary     0.543    10 0.0221  Preproces~
##  3    0.0000000178          1 accuracy binary     0.665    10 0.00816 Preproces~
##  4    0.0000000178          1 roc_auc  binary     0.543    10 0.0221  Preproces~
##  5    0.00000316            1 accuracy binary     0.665    10 0.00816 Preproces~
##  6    0.00000316            1 roc_auc  binary     0.543    10 0.0221  Preproces~
##  7    0.000562              1 accuracy binary     0.665    10 0.00816 Preproces~
##  8    0.000562              1 roc_auc  binary     0.543    10 0.0221  Preproces~
##  9    0.1                   1 accuracy binary     0.668    10 0.00926 Preproces~
## 10    0.1                   1 roc_auc  binary     0.5      10 0       Preproces~
## # ... with 40 more rows
```

Despite the Tuning, the model is still not good. Although the ROC estimate has risen (0.5), the accuracy is even lower than before (~0.66). In order to understand what happened in the process that led to this result, we'll plot the performance:

```r
# plot performance
tree_stat_res  %>%
  collect_metrics() %>%
  mutate(tree_depth = factor(tree_depth)) %>%
  ggplot(aes(cost_complexity, mean, color = tree_depth)) +
  geom_line(size = 1.5, alpha = 0.6) +
  geom_point(size = 2) +
  facet_wrap(~ .metric, scales = "free", nrow = 2) +
  scale_x_log10(labels = scales::label_number()) +
  scale_color_viridis_d(option = "plasma", begin = .9, end = 0)
```



The accuracy was the best when the decision tree was of depth 4, and accordingly the cost complexity is high. Let's use the show_best() function to see the top 5 candidate models by default:

```
# best tree
tree_stat_res %>%
  show_best("roc_auc")
```

```
## # A tibble: 5 x 8
##   cost_complexity tree_depth .metric .estimator  mean     n std_err .config
##             <dbl>      <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    0.0000000001          4 roc_auc binary     0.651    10  0.0243 Preprocesso~
## 2    0.0000000178          4 roc_auc binary     0.651    10  0.0243 Preprocesso~
## 3    0.00000316            4 roc_auc binary     0.651    10  0.0243 Preprocesso~
## 4    0.000562              4 roc_auc binary     0.651    10  0.0243 Preprocesso~
## 5    0.0000000001          8 roc_auc binary     0.643    10  0.0251 Preprocesso~
```

The first 4 best models are of depth 4 and one of 8. But which is the best? Workflow can determine that as follows:

```
best_tree <- tree_stat_res %>%
  select_best("roc_auc") # Selecting the best tree

best_tree
```

```
## # A tibble: 1 x 3
```

```
##   cost_complexity tree_depth .config
##             <dbl>      <int> <chr>
## 1    0.0000000001          4 Preprocessor1_Model06
```

```
# Finalize workflow
final_wf <-
  tree_stat_wf %>%
  finalize_workflow(best_tree)

final_wf
```

```
## == Workflow ====================================================================
## Preprocessor: Formula
## Model: decision_tree()
##
## -- Preprocessor ----------------------------------------------------------------
## status ~ .
##
## -- Model -----------------------------------------------------------------------
## Decision Tree Model Specification (classification)
##
## Main Arguments:
##   cost_complexity = 1e-10
##   tree_depth = 4
##
## Computational engine: rpart
```

Now that we found the best tree, let's look who is this tree:

```
final_tree <-
  final_wf %>%
  fit(data = ms_train_stat)

final_tree
```

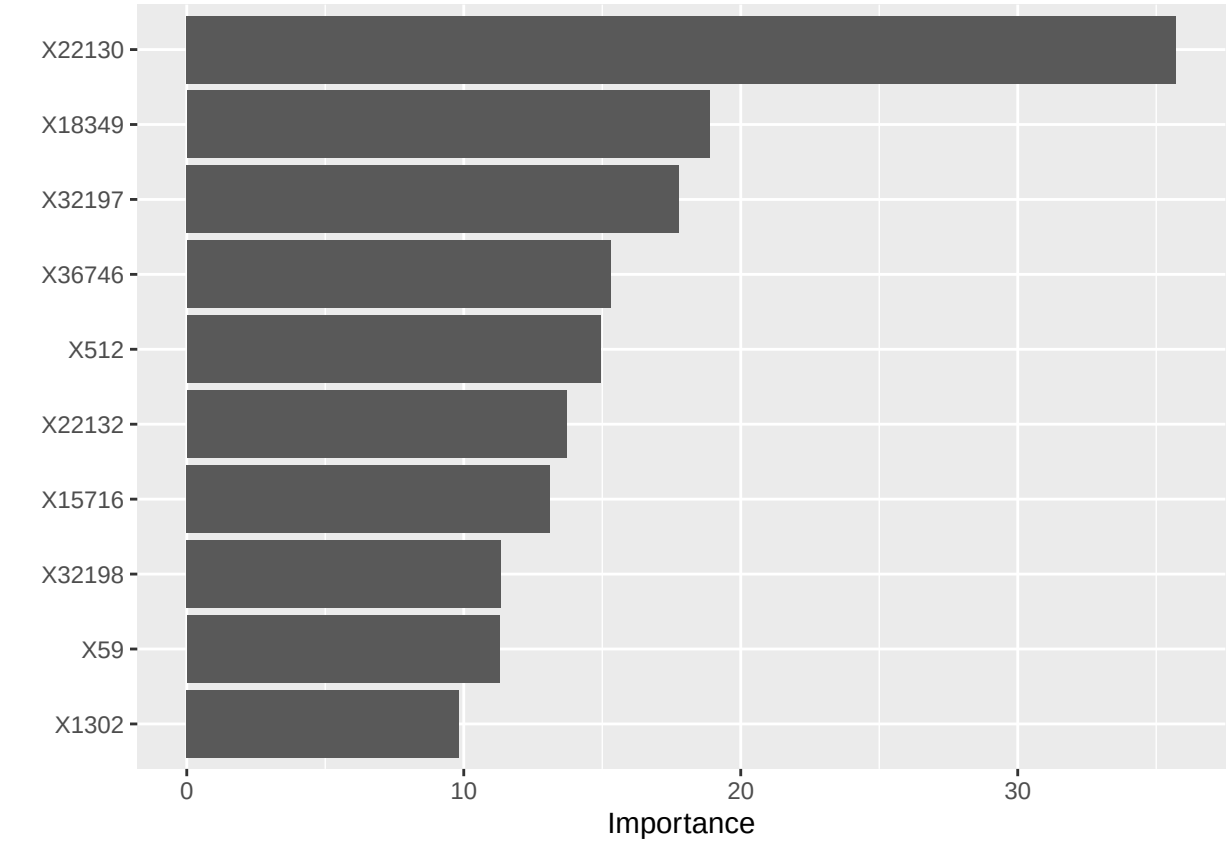
```
## == Workflow [trained] ==========================================================
## Preprocessor: Formula
## Model: decision_tree()
##
## -- Preprocessor ----------------------------------------------------------------
## status ~ .
##
## -- Model -----------------------------------------------------------------------
## n= 714
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##  1) root 714 237 MS (0.3319328 0.6680672)
##    2) X22130>=14.02517 366 177 MS (0.4836066 0.5163934)
##      4) X512>=19.19939 190  72 HC (0.6210526 0.3789474)
##        8) X32198< 19.73003 166  52 HC (0.6867470 0.3132530)
##         16) X46115< 14.49413 150  39 HC (0.7400000 0.2600000) *
```

```
##         17) X46115>=14.49413 16   3 MS (0.1875000 0.8125000) *
##        9) X32198>=19.73003 24   4 MS (0.1666667 0.8333333)
##         18) X1649>=19.73984 7   3 HC (0.5714286 0.4285714) *
##         19) X1649< 19.73984 17   0 MS (0.0000000 1.0000000) *
##      5) X512< 19.19939 176  59 MS (0.3352273 0.6647727)
##       10) X46957< 15.99291 105  48 MS (0.4571429 0.5428571)
##         20) X22036>=13.76816 24   5 HC (0.7916667 0.2083333) *
##         21) X22036< 13.76816 81  29 MS (0.3580247 0.6419753) *
##       11) X46957>=15.99291 71  11 MS (0.1549296 0.8450704) *
##    3) X22130< 14.02517 348  60 MS (0.1724138 0.8275862)
##      6) X43488>=14.53925 67  25 MS (0.3731343 0.6268657)
##       12) X59>=18.05973 40  18 HC (0.5500000 0.4500000)
##         24) X34445< 14.82395 23   5 HC (0.7826087 0.2173913) *
##         25) X34445>=14.82395 17   4 MS (0.2352941 0.7647059) *
##       13) X59< 18.05973 27   3 MS (0.1111111 0.8888889) *
##      7) X43488< 14.53925 281  35 MS (0.1245552 0.8754448) *
```

The feature that impacted on the status of disease classification we can use the vip package as follows:

```
final_tree %>%
  pull_workflow_fit() %>%
  vip()
```

```
## Warning: `pull_workflow_fit()` was deprecated in workflows 0.2.3.
## Please use `extract_fit_parsnip()` instead.
```



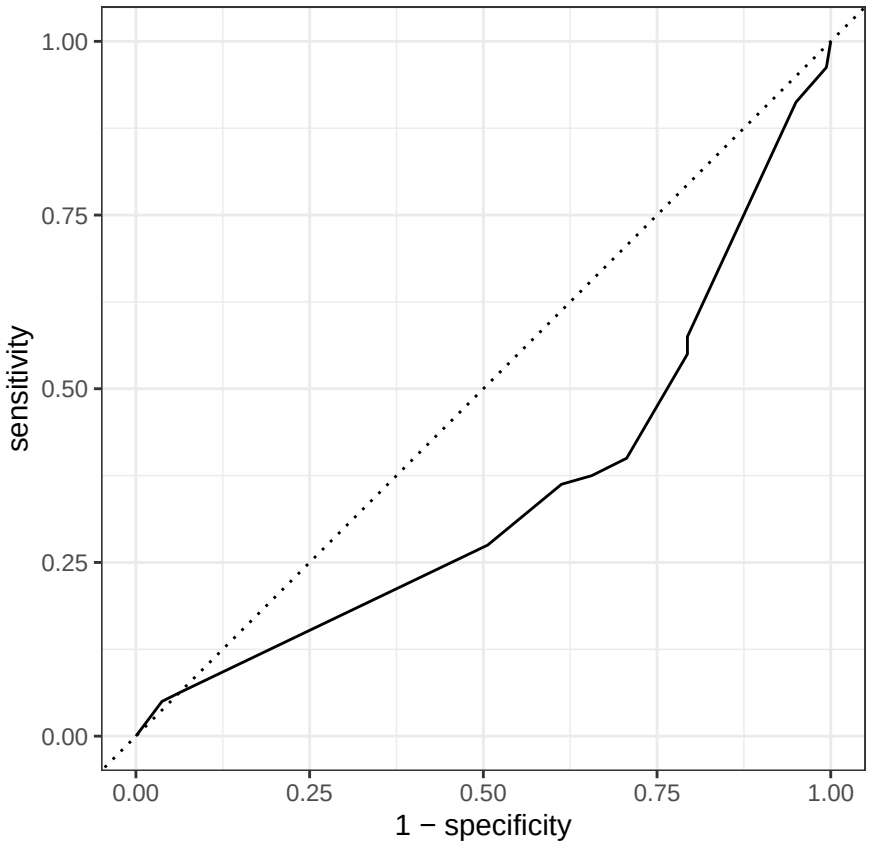
The graph indicates that the main features affecting the classification in our model are (in descending order): 1. Phenyllactate (PLA), a part of phenylalanine catabolism, mainly associated with the Mendelian disease phenylketonuria (PKU). 2. Indolelactate, a tryptophan metabolite (that was previously associated with MS, levi et al. 2021). 3. 3-(4-hydroxyphenyl)lactate, a tyrosine metabolite. 4. 2-hydroxy-3-methylvalerate, an organic acid generated by L-isoleucine metabolism, associated with Maple Syrup Urine Disease (MSUD). 5. Asparagine, an amino acid. 6. Alpha-hydroxyisocaproate, an end product of leucine metabolism. 7. Imidazole lactate, a component of a normal human urine. 8. Acetylcarnitine, the shortest form of acylcarnitine that participates in brain metabolism due to likely transport across the blood–brain barrier. 9. Histidine, an amino acid. 10. Methionine, an amino acid. These results are interesting biological wise. However, we need to check the trade off between sensitivity and specificity to understand the validity of the model. For this purpose, we'll plot the ROC curve:

```
# Set workflow
final_fit_stat <-
  final_wf %>%
  last_fit(ms_split)

# train
final_fit_stat %>%
  collect_metrics()
```

```
## # A tibble: 2 x 4
##   .metric  .estimator .estimate .config
##   <chr>    <chr>          <dbl> <chr>
## 1 accuracy binary         0.679 Preprocessor1_Model1
## 2 roc_auc  binary         0.648 Preprocessor1_Model1
```

```
# collect metrics
final_fit_stat %>%
  collect_predictions() %>%
  roc_curve(status, .pred_MS) %>%
  autoplot()
```




```
args(decision_tree)
```

```
## function (mode = "unknown", engine = "rpart", cost_complexity = NULL,
##     tree_depth = NULL, min_n = NULL)
## NULL
```

The ROC curves show that the model was more specific than sensitive in its classification process. This further strengthen the notion of the lack of validity of the classification of health status. To sum up this part, the classification model depending on the status of health didn't show in the overall analysis any reliability in good classification. To further assert whether this is specific to the random forest or not, in this report I'll check the classification with random forest on the other factors and other classification algorithms. By that we can also see if the results above are reproducible and significant.

Summary

In this report we demonstrate how by metabolic data and three known factors: status of health, gender and race using 4 classification models: K-Nearest Neighbors (KNN), K-means, random forest and Support Vector Machine (SVM); and the feature reduction model T-distributed Stochastic Neighbor Embedding (t-SNE). The best classification was achieved when classifying by gender model using the random forest algorithm, while the worst was with KNN for the status model. In the most affecting metabolite area, the results were not conclusive. The results of Fitzgerald et al 2021 showed, came in agreement with the classification of MS in the status of health in the random forest, which had about 70% accuracy. On the other hand, the best and the most valid classification model we had, random forest by gender, indicated on other amino acids precursors and hormones not specifically associated with aromatic amino acids metabolism.

Findings

The K-Nearest Neighbors (KNN) model

The simplest model to perform, just to get a clear picture on the model is the KNN model. The first step in this model is to normalize the data. The min-max normalization strategy will suffice.

```
# Normalization function
normalize <- function(x) {
num <- x - min(x)
denom <- max(x) - min(x)
return (num/denom)
}

# Normalizing the data
msnorm <- as.data.frame(lapply(ms_dat[7:275], normalize))
```

After normalization, we can perform the KNN algorithm using the caret package. First we'll choose the effect of the status of health:

```
# Splitting the data with a novel index
index <- createDataPartition(ms_dat$status, p=0.8, list=FALSE)

# Subset training set with the index
statmsnorm.training <- msnorm[index,]
statms.training <- ms_dat[index,]

# Subset test set with the index
statmsnorm.test <- msnorm[-index,]
statms.test <- ms_dat[-index,]

# Train a model
statmodel_knn <- train(statmsnorm.training[, 5:269], statms.training[,2], method='knn')

statmodel_cart <- train(statmsnorm.training[, 5:269], statms.training[, 2], method='rpart2')
```

Prediction using the KNN model produces the following results:

```
# Predict the labels of the test set
predictions<-predict.train(object=statmodel_knn,statmsnorm.test[,5:269], type="raw")

# Evaluate the predictions
table(predictions)
```

```
## predictions
##  HC  MS
##  67 123
```

```
# Confusion matrix
confusionMatrix(predictions,statms.test[,2])
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction HC MS
##         HC 29 38
##         MS 34 89
##
##                Accuracy : 0.6211
##                  95% CI : (0.548, 0.6903)
##     No Information Rate : 0.6684
##     P-Value [Acc > NIR] : 0.9272
##
##                   Kappa : 0.1586
##
##  Mcnemar's Test P-Value : 0.7237
##
##             Sensitivity : 0.4603
##             Specificity : 0.7008
##          Pos Pred Value : 0.4328
##          Neg Pred Value : 0.7236
##              Prevalence : 0.3316
##          Detection Rate : 0.1526
##    Detection Prevalence : 0.3526
##       Balanced Accuracy : 0.5806
##
##        'Positive' Class : HC
##
```

The accuracy of prediction in the status model is not good, 0.62. The sensitivity, specificity and the ability to predict false values (negative and positive) also indicate that the model's efficiency is low. Moreover, the detection parameters and prevalence are low (approximately 0.3-0.2). Changing the K (data not shown) didn't show any improvement in classification... But it can make sense since finding a person of MS randomly from many healthy individuals is not always accurate because the frequency of the disease is not high in the population. Hence, let's try the KNN model on the gender factor:

```
# Splitting the data with a novel index
index <- createDataPartition(ms_dat$gender, p=0.8, list=FALSE)

# Subset training set with the index
genmsnorm.training <- msnorm[index,]
genms.training <- ms_dat[index,]

# Subset test set with the index
genmsnorm.test <- msnorm[-index,]
genms.test <- ms_dat[-index,]

# Train a model
genmodel_knn <- train(genmsnorm.training[, 5:269], genms.training[,4], method='knn')

genmodel_cart <- train(genmsnorm.training[, 5:269], genms.training[, 4], method='rpart2')
```

Prediction using the KNN model produces the following results:

```r
# Predict the labels of the test set
predictions<-predict.train(object=genmodel_knn,genmsnorm.test[,5:269], type="raw")

# Evaluate the predictions
table(predictions)
```

```
## predictions
##   F   M
## 157  33
```

```r
# Confusion matrix
confusionMatrix(predictions,genms.test[,4])
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   F   M
##          F 128  29
##          M  10  23
##
##                Accuracy : 0.7947
##                  95% CI : (0.7303, 0.8498)
##     No Information Rate : 0.7263
##     P-Value [Acc > NIR] : 0.018810
##
##                   Kappa : 0.4174
##
##  Mcnemar's Test P-Value : 0.003948
##
##             Sensitivity : 0.9275
##             Specificity : 0.4423
##          Pos Pred Value : 0.8153
##          Neg Pred Value : 0.6970
##              Prevalence : 0.7263
##          Detection Rate : 0.6737
##    Detection Prevalence : 0.8263
##       Balanced Accuracy : 0.6849
##
##        'Positive' Class : F
##
```

In contrast to what was observed in the status classification model, the accuracy of prediction in the gender model is good, 0.8. The sensitivity, specificity and the ability to predict false values (negative and positive) show the model's efficiency is good as well as the detection prevalence. However, detection rate and prevalence are not so good (approximately 0.7). Changing the K didn't show any improvement in classification (data not shown), what leads us to suggest that finding a female with MS among healthy women is more probable since the disease's frequency in females is much higher than in men. Finally, classifying the model according to the race factor didn't work because of the imbalance between the white ethnicity and the rest. What leads us to the conclusion of this part that the only model that was efficient was the gender KNN model. However, it can be just in the KNN algorithm. We'll need to test these conclusions in additional models.

K-means

K-means has some similarities to KNN, for example a fixed number for the analysis. In this analysis this parameter K specifies the desired number of clusters. Moreover the kmeans() function requires a data frame containing only numeric data in addition to K , Hence:

```
ms_knum <- ms_dat[7:275]
```

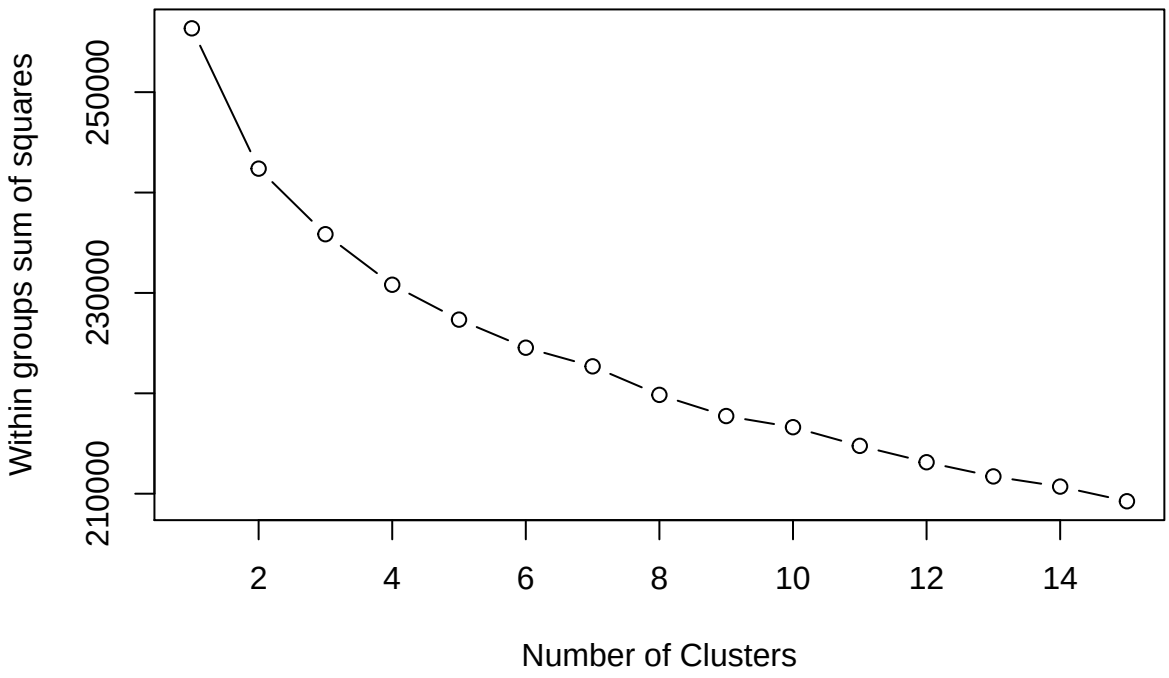
Just as in KNN, kmeans is sensitive to the distribution of our data, so we'll use z-score standardization on our data.

```
# Coerced back to data frame form using the as.data.frame() function.
ms_dat_z <- as.data.frame(lapply(ms_knum, scale))
```

With that in hand, we need to determine the K value for it can't be too specific to be useful or too small resulting in heterogeneous groupings. For this purpose, we will use an elbow plot to see what seems to be the best k and see how it goes.

```
wssplot <- function(data, num_clusters=15, seed=1234){
  wss <- (nrow(data)-1)*sum(apply(data,2,var))
  for (i in 2:num_clusters){
    set.seed(seed)
    wss[i] <- sum(kmeans(data, centers=i)$withinss)}
  plot(1:num_clusters, wss, type="b", xlab="Number of Clusters",
       ylab="Within groups sum of squares")
  wss
}

wssplot(ms_dat_z)
```




```
##  [1] 256357.0 242382.8 235849.7 230817.3 227340.9 224544.9 222688.9 219844.1
##  [9] 217736.8 216625.8 214766.7 213136.1 211720.8 210708.8 209249.7
```

This elbow plot is not clear due to the fact that there isn't a clear "elbow". Let's start with k=3 with nstart parameter a value of 25 (25 configurations) and report the best one.

```
set.seed(2345)
ms_clusters_3 <- kmeans(ms_dat_z, centers = 3, nstart = 25)
ms_clusters_3
```

```
## K-means clustering with 3 clusters of sizes 340, 318, 296
##
## Cluster means:
##        X48762      X20675     X34400     X30460     X32350      X34395
## 1 -0.06422284  0.07824466  0.1175586  0.1235642  0.0145032  0.05430174
## 2  0.28391238  0.05130575  0.1333852  0.3503256  0.3363713  0.25142011
## 3 -0.23124450 -0.14499464 -0.2783325 -0.5182952 -0.3780310 -0.33248036
##        X34389     X38395      X38168     X38276     X42374       X43761
## 1  0.08091412 -0.1131685  0.12339046 -0.1686836  0.1275027 -0.007750306
## 2  0.15571319  0.2366157 -0.05161661  0.4812466  0.1800335  0.131654171
## 3 -0.26022836 -0.1242112 -0.08627930 -0.3232567 -0.3398702 -0.132536900
##        X43343     X43266     X36746      X42489      X18281      X22036
## 1  0.16137588 -0.1076417 -0.1357054 -0.02889891  0.06369133 -0.07293521
## 2  0.06935773  0.2616462  0.4193648  0.28429373  0.11752988  0.26376110
## 3 -0.25987689 -0.1574504 -0.2946559 -0.27222897 -0.19942417 -0.19958803
```

```
##       X35675     X45095      X46115     X32197      X31787     X32397
## 1  0.5338745 -0.1151587  0.20823203 -0.1293357  0.06724955  0.2989560
## 2 -0.2607889  0.7378292  0.03891728  0.6015781  0.17137358  0.2854427
## 3 -0.3330624 -0.6603910 -0.28099523 -0.4977288 -0.26135691 -0.6500535
##         X531       X542     X22053      X39600        X1549     X32457
## 1 -0.1128784  0.6995711  0.6700371 -0.07449572 -0.001298525  0.7333965
## 2  0.4856402 -0.4346138 -0.3532384  0.29466770  0.406249303 -0.4768824
## 3 -0.3920775 -0.3366453 -0.3901446 -0.23099928 -0.434951959 -0.3300885
##        X22001      X48448      X27672      X48763      X12017      X46165
## 1  0.41282407 -0.08440383 -0.01648414 -0.02987081  0.14481765 -0.07607491
## 2 -0.09856386  0.39227494  0.26746820  0.31804968 -0.11603060  0.39624111
## 3 -0.36830025 -0.32448016 -0.26841310 -0.30737745 -0.04169011 -0.33830812
##         X46548      X15677      X32445      X15749       X1558      X37181
## 1  0.055110739 -0.07829794  0.08745122 -0.01495762  0.08859445 -0.01213498
## 2  0.009233855  0.28500121  0.08420683  0.14715015  0.29111845  0.17724059
## 3 -0.073223031 -0.21624691 -0.19091617 -0.14090594 -0.41451952 -0.17647505
##        X36099      X48441     X35527      X1669      X46146      X36098
## 1  0.01901998  0.12060802 -0.1965808 -0.2970245  0.03603129 -0.03421133
## 2  0.18993296  0.05590331  0.3518512  0.5061023  0.19630763  0.22451540
## 3 -0.22589688 -0.19859453 -0.1522000 -0.2025412 -0.25228535 -0.20190556
##         X1418     X34424     X35136       X1494      X37186     X37190
## 1  0.29018576  0.1129487  0.2035533 -0.04760042  0.20866561  0.1619752
## 2 -0.03428385  0.1782172  0.1604114  0.36974757 -0.02693792  0.1453599
## 3 -0.29648952 -0.3212015 -0.4061452 -0.34255264 -0.21074341 -0.3422162
##        X37198      X37200      X37196     X43231      X36776     X35114
## 1  0.19780704  0.14366708  0.15234552 -0.2721305  0.29319547  0.2419987
## 2 -0.20792490 -0.22265578 -0.24965064  0.4524763 -0.06746307  0.1588999
## 3 -0.00383201  0.07418152  0.09321427 -0.1735240 -0.26430137 -0.4486814
##        X34390     X32198      X43258      X1126      X22132      X32827
## 1  0.06629039  0.5842342  0.25376914 -0.3702715 -0.03920923  0.09999208
## 2  0.13274673 -0.1358697  0.01747573  0.5361171  0.33744171  0.05741881
## 3 -0.21875740 -0.5251117 -0.31026618 -0.1506518 -0.31748420 -0.17654220
##        X37209      X37202      X37203      X37211      X37210       X31591
## 1  0.13272419  0.11768436  0.15327724  0.12408787  0.14003304  0.168531969
## 2  0.04611363  0.09245007  0.08461933  0.03273167 -0.02593566 -0.001653702
## 3 -0.20199445 -0.23449934 -0.26697030 -0.17769779 -0.13298545 -0.191807406
##         X1638       X512      X12129       X3141     X32586     X43807
## 1 -0.14565450 -0.2017001 -0.08370539 -0.07760431  0.4289949  0.2421536
## 2  0.11186141  0.4012144  0.33312390  0.32536623 -0.2524214 -0.1046087
## 3  0.04713041 -0.1993518 -0.26173502 -0.26040876 -0.2215820 -0.1657657
##         X2137     X32412     X48782        X569       X1642      X32492
## 1  0.22066822  0.2010342  0.3079440 -0.03840533  0.20060162 -0.08663567
## 2 -0.05018747  0.1911287  0.1617739  0.22436448  0.01434921  0.24829522
## 3 -0.19955263 -0.4362520 -0.5275171 -0.19692599 -0.24583649 -0.16723565
##        X15500      X35320       X1563      X22842     X15506      X34396
## 1  0.05487651 -0.07411583 -0.13400783 -0.20935116 -0.1173176 -0.10996287
## 2  0.29412492  0.41136671  0.19961567  0.21306773  0.4266537  0.15638979
## 3 -0.37901939 -0.35680821 -0.06052406  0.01156709 -0.3236077 -0.04170465
##        X38637     X38178       X1564      X2132        X1712         X1769
## 1  0.20041536  0.5967287  0.22133218  0.2021631 -0.006192259  0.0945527585
## 2 -0.07598165 -0.3075769 -0.05470647  0.1395019 -0.028986032 -0.0005627265
## 3 -0.14857790 -0.3549942 -0.19546042 -0.3820848  0.038253129 -0.1080033476
##        X27718        X513      X18368      X22176     X33941      X32425
## 1 -0.13267371  0.04392907  0.15283658 -0.03054590  0.6514952  0.11601336
```

```
## 2  0.10505971  0.39950468  0.04932769  0.10859783 -0.3194089 -0.03092314
## 3  0.03952728 -0.47965666 -0.22854947 -0.08158279 -0.4051903 -0.10003712
##        X36747       X1114     X36808      X5086      X32388     X48407
## 1  0.09892055 -0.12656474  0.2510693 -0.1294992  0.33173930 -0.2549794
## 2  0.36712902  0.17440027  0.3163879  0.4077225 -0.04865573  0.4074949
## 3 -0.50804059 -0.04198403 -0.6282936 -0.2892771 -0.32877987 -0.1448999
##        X39831      X33973     X37459       X20699      X42420     X15765
## 1  0.09459998  0.19274804 -0.0847643 -0.003100504 -0.06415884  0.2569300
## 2 -0.04853056  0.03347894  0.2567612  0.209495134  0.45799407  0.0984483
## 3 -0.05652457 -0.25736701 -0.1784804 -0.221504328 -0.41833820 -0.4008878
##        X47112      X1643      X44876      X36738        X2730     X18369
## 1  0.21320646  0.1539611 -0.02025810 -0.05487228  0.144901033 -0.2814806
## 2  0.01067472  0.2020277  0.07400714  0.37424308  0.004101452  0.7051899
## 3 -0.25636742 -0.3938905 -0.05623823 -0.33902947 -0.170846666 -0.4342804
##       X44872     X33422      X2734     X43829      X18280        X587
## 1 -0.2710011 -0.2440409 -0.3295904 -0.1802034 -0.05426179 -0.02930728
## 2  0.4867435  0.7668102  0.6853373  0.5141231  0.30846887  0.24544670
## 3 -0.2116352 -0.5434855 -0.3576909 -0.3453446 -0.26906788 -0.23002559
##        X15443        X57        X53        X396       X1572     X15122
## 1  0.08179455 -0.1761615  0.1365931 -0.02107685 -0.12225700  0.4717841
## 2  0.21174913  0.4474510  0.0155388  0.07495069  0.22374249 -0.3293462
## 3 -0.32144044 -0.2783598 -0.1735912 -0.05631146 -0.09994166 -0.1880895
##        X37455         X58      X32346      X18476      X18477      X42574
## 1 -0.09081194  0.06861837 -0.45569687 -0.45163392 -0.30548576 -0.17053373
## 2  0.19941707  0.04534942  0.51594741  0.50429543  0.37575768  0.20012760
## 3 -0.10992760 -0.12753839 -0.03085926 -0.02300816 -0.05278981 -0.01911861
##        X32620      X39379      X43802     X46957     X15753         X59
## 1  0.04434609 -0.24570873  0.05872484  0.0619498  0.0289061 -0.26727948
## 2  0.14422071  0.19886522  0.07170169  0.2762253  0.2668385  0.35354797
## 3 -0.20587789  0.06858726 -0.14448508 -0.3679141 -0.3198740 -0.07281497
##         X22137      X22138      X33009     X35322        X15716     X27513
## 1 -0.166582704 -0.08143267 -0.03041363 -0.1357985  0.0007334354 -0.0653145
## 2  0.176552238  0.34860005  0.30559157  0.2912646  0.1810439296  0.3170192
## 3  0.001670634 -0.28097198 -0.29336988 -0.1569279 -0.1953423569 -0.2655580
##       X18349      X32405     X33441      X1125     X44656     X34407
## 1 -0.1672619 -0.08815876 -0.2453217 -0.3597139 -0.1556080 -0.2500200
## 2  0.5302739  0.16048473  0.5964466  0.7198881  0.3217706  0.6579881
## 3 -0.3775609 -0.07114921 -0.3589887 -0.3602084 -0.1669470 -0.4197075
##         X1417     X15140        X527     X34534        X60       X46223
## 1  0.02986515 -0.0724639 -0.07686713  0.6949389 -0.2878324  0.424354698
## 2  0.45723914  0.4538038  0.29855916 -0.2652568  0.7018323  0.006595625
## 3 -0.52552770 -0.4042969 -0.23245605 -0.5132687 -0.4233772 -0.494520291
##        X1301       X1303     X20676      X1302     X44878     X18374
## 1 -0.1432693  0.02171187 -0.0973870 -0.2767786  0.1402169 -0.4327489
## 2  0.4807360  0.35420938  0.3086458  0.4779755  0.2202447  0.5860827
## 3 -0.3519003 -0.40547506 -0.2197223 -0.1955793 -0.3976743 -0.1325665
##        X46144      X48429       X1496      X15745     X48182      X33952
## 1  0.03138707  0.01012153 -0.07599793  0.08374332  0.7624754  0.59097487
## 2  0.09594278 -0.09783090  0.26801900  0.35483808 -0.3531544 -0.06850661
## 3 -0.13912637  0.09347604 -0.20064442 -0.47740283 -0.4964140 -0.60522417
##       X31536       X37432      X1585      X43488     X15720      X33943
## 1 -0.1080619  0.005346199  0.2152853 -0.07190302 -0.1981408 -0.06145296
## 2  0.3499976  0.160884366  0.3551810  0.40407859  0.5392429  0.22239699
## 3 -0.2518858 -0.178982892 -0.6288667 -0.35152015 -0.3517276 -0.16833864
```

```
##       X27710      X1587       X1589     X33950      X37496      X48187
## 1  0.4347294 -0.3504952  0.09394621 -0.2436165  0.08487757  0.05218455
## 2 -0.2694960  0.6897180  0.10498230  0.6021255  0.14586220  0.32494894
## 3 -0.2098253 -0.3383850 -0.22069622 -0.3670483 -0.25419781 -0.40904226
##       X32390       X1591      X43249      X2829     X47101      X37431
## 1 -0.3228287 -0.07476923  0.04791808  0.1508447  0.1122130 -0.07188648
## 2  0.5318301  0.55581163  0.35491557  0.2333320  0.2535008  0.14088141
## 3 -0.2005412 -0.51123838 -0.43633547 -0.4239418 -0.4012354 -0.06878002
##       X42092      X40469     X35137       X1498        X594      X46111
## 1  0.5809749  0.00976497  0.2909402 -0.03130915  0.05483585 -0.01374473
## 2 -0.3454959  0.18059700  0.1482158  0.39943520  0.08573148  0.42771478
## 3 -0.2961614 -0.20523627 -0.4934200 -0.39315974 -0.15509054 -0.44371653
##       X45413     X33936     X38102      X35160      X1493       X1505
## 1  0.3217154  0.6311701  0.5065993  0.58582360 -0.0752518 -0.01649309
## 2  0.1445339 -0.3164767 -0.2318687 -0.06787527  0.5338345  0.28274754
## 3 -0.5248142 -0.3849941 -0.3328024 -0.59998544 -0.4870735 -0.28481779
##         X20694      X36103       X48841      X44681      X1508      X18254
## 1  0.006993539  0.11706613  0.117481340  0.53574340  0.1305566  0.08353781
## 2 -0.033762591  0.05644077 -0.005953912 -0.01622851  0.1664896  0.21254091
## 3  0.028238854 -0.19510355 -0.128548350 -0.59794625 -0.3288275 -0.32429346
##        X32553     X35126        X64     X22130       X566       X1444
## 1  0.01839581  0.1043324 -0.3153681 -0.2502402 -0.3557832 -0.16520601
## 2  0.12260529  0.1175992  0.6960238  0.5598231  0.4440773  0.22263930
## 3 -0.15284816 -0.2461809 -0.3855082 -0.3139935 -0.0684131 -0.04942317
##         X33935      X32619      X40708      X32562      X38170      X35127
## 1 -0.123799736  0.17632033  0.14954045  0.16928742  0.05990301  0.03609341
## 2  0.140692026 -0.01761116 -0.20211191  0.02722242  0.02239364  0.19527829
## 3 -0.008946466 -0.18361001  0.04536431 -0.22369747 -0.09286554 -0.25125086
##        X1898     X40703     X32452     X33442      X31555      X46225
## 1 -0.4287981 -0.3450060 -0.3094299  0.3470367 -0.03417513  0.06184583
## 2  0.6557010  0.6545278  0.7132047  0.1632167  0.18698211  0.19541883
## 3 -0.2118972 -0.3068844 -0.4107869 -0.5739709 -0.16162422 -0.28098233
##        X18335      X15772      X43239      X39592       X1515       X1648
## 1  0.05099704 -0.08646601 -0.06310597  0.09481972 -0.009550266 -0.08241071
## 2  0.24287259  0.29664199  0.13833332 -0.04262190  0.105662971  0.20457687
## 3 -0.31950161 -0.21937064 -0.07612826 -0.06312481 -0.102546400 -0.12512096
##        X17747      X34445      X34384     X34409      X1437     X37058
## 1 -0.01207922  0.03988515 -0.06286542  0.4062853  0.1717612 -0.1703405
## 2  0.22151545  0.20474008  0.13073160  0.1059629  0.1763289  0.4406877
## 3 -0.22410465 -0.26577128 -0.06823786 -0.5805175 -0.3867277 -0.2777802
##        X46960      X20693      X18494     X18497      X32807      X12261
## 1 -0.07615481 -0.05824253 -0.39703410 -0.2346582  0.08169608 -0.34626743
## 2  0.38056050  0.02606232  0.46928265  0.3598661  0.07424860  0.42010098
## 3 -0.32137028  0.03890082 -0.04810908 -0.1170731 -0.17360717 -0.05358509
##        X36850       X18392      X18394       X27738      X1284      X36095
## 1  0.00179888 -0.003617972  0.08278957  0.002338803 -0.1812388 -0.08841989
## 2  0.18721333  0.127398118  0.15280397  0.034040003  0.2953086  0.18280652
## 3 -0.20319411 -0.132711119 -0.25925716 -0.039256466 -0.1090775 -0.09483010
##         X2761      X35428      X32306        X607      X32401        X54
## 1  0.09335643 -0.02006478 -0.10610638  0.07601996 -0.02393891 -0.3828972
## 2 -0.13926434  0.46856686  0.17758555 -0.09779270  0.33915601  0.5363142
## 3  0.04238134 -0.48034538 -0.06890552  0.01774085 -0.33686616 -0.1363610
##         X37097      X48408      X1299       X1604       X1670        X606
## 1  0.001823095 -0.12640014 -0.4219960  0.1178933 -0.09445994  0.34697229
```

```
## 2  0.096579965  0.17284204  0.7062517  0.3440941  0.46698024 -0.09795502
## 3 -0.105852301 -0.04049906 -0.2740183 -0.5050867 -0.39318696 -0.29331378
##          X1605      X1649      X1567       X3147     X15679
## 1  0.005493032 -0.3281201  0.1134067  0.10147978 -0.2337830
## 2 -0.029911165  0.7118643  0.3032206  0.08053747  0.4566019
## 3  0.025824728 -0.3878785 -0.4560217 -0.20308798 -0.2220040
##
## Clustering vector:
##   [1] 3 1 1 2 1 3 1 1 1 3 2 2 2 3 1 3 1 1 3 3 3 1 3 2 1 1 3 2 2 3 3 1 2 3 1 2 2
##  [38] 3 2 1 3 1 3 1 1 3 1 3 1 1 1 3 1 3 2 2 3 1 1 1 3 2 3 3 3 1 1 2 2 2 3 3 1 3
##  [75] 1 1 3 1 1 3 3 3 1 2 2 2 3 3 3 1 2 1 1 3 1 1 1 1 2 3 2 3 1 1 3 3 1 2 2 2 2
## [112] 2 1 1 1 3 3 2 3 2 2 3 1 2 1 1 1 2 1 1 1 3 3 3 2 2 3 1 1 3 3 1 2 1 1 1 1 1
## [149] 1 3 1 1 1 3 1 3 2 3 3 3 3 2 2 3 3 3 2 2 3 1 1 1 3 3 3 2 1 1 1 2 1 1 1 3 2
## [186] 3 2 3 1 1 3 2 2 2 3 2 1 2 3 2 2 3 3 3 2 3 2 2 1 2 1 2 3 3 3 2 1 3 2 2 1 1
## [223] 3 1 3 3 3 3 2 1 1 2 3 1 2 3 3 2 2 1 3 2 2 2 1 1 3 1 1 1 2 3 2 2 2 2 3 2 3
## [260] 2 1 2 3 2 1 3 1 1 1 3 1 3 3 3 1 3 3 2 3 3 1 1 1 2 2 2 2 1 1 1 1 3 3 2 3 2
## [297] 1 1 1 2 2 2 3 3 2 3 3 2 2 2 3 1 1 1 3 3 1 2 3 1 1 2 2 2 1 3 2 2 2 2 2 3 1
## [334] 1 3 2 3 1 1 1 3 2 1 1 2 2 3 1 2 3 2 2 1 2 3 2 1 1 3 1 2 2 2 3 1 3 1 1 2 2
## [371] 2 3 3 1 3 2 1 1 3 1 1 3 3 2 1 1 2 2 2 1 2 2 3 3 1 1 2 1 3 3 1 3 1 3 2 3 1
## [408] 1 2 1 3 1 2 3 2 2 2 1 1 2 2 2 1 1 2 1 1 1 2 2 2 3 3 2 2 3 1 2 2 1 2 1 1 1
## [445] 3 3 1 1 2 1 1 3 2 1 1 1 3 1 1 2 1 3 3 3 2 1 3 2 1 3 1 1 1 2 3 2 2 1 3 3 2
## [482] 1 1 1 1 2 3 3 3 1 1 2 3 2 2 1 2 1 3 3 1 2 3 2 2 2 2 2 1 2 3 3 3 2 1 1 2 2
## [519] 3 1 3 2 1 1 1 3 3 3 3 3 3 3 3 2 3 2 2 2 3 1 1 2 1 1 2 1 2 2 2 1 3 3 3 3 3
## [556] 3 3 1 2 2 2 2 2 3 3 2 1 1 1 1 2 2 2 2 2 1 2 1 3 1 1 1 2 1 2 1 1 1 1 2 1 2
## [593] 2 2 1 2 2 1 1 1 1 1 2 3 2 1 3 3 3 3 3 1 2 3 1 1 2 2 2 2 1 2 1 1 2 2 2 1 2
## [630] 3 2 1 2 2 2 1 1 1 1 1 2 1 3 2 1 3 2 2 1 1 2 2 2 3 2 2 1 2 3 2 3 1 3 3 2 3
## [667] 2 1 2 3 2 3 3 3 3 3 3 1 1 1 3 2 1 3 1 2 2 2 3 2 3 1 1 1 2 1 3 3 2 1 3 2 3
## [704] 1 1 1 1 1 3 1 3 1 3 3 2 1 1 3 1 1 3 3 1 2 3 2 3 3 1 3 1 3 1 2 2 2 2 2 1 1
## [741] 2 3 3 1 3 2 1 3 3 1 3 1 3 3 1 1 3 2 3 2 3 1 3 2 2 1 3 1 1 3 1 3 1 3 1 2 1
## [778] 1 2 2 3 2 1 2 2 2 1 2 3 3 1 3 2 1 2 3 1 1 3 3 3 3 2 2 3 3 3 1 3 3 2 3 1 1
## [815] 1 3 3 2 1 1 3 2 1 1 3 2 3 3 1 2 2 2 2 2 2 2 1 3 3 1 2 2 3 1 2 2 3 1 2 1 3
## [852] 1 2 1 2 2 2 1 2 1 2 3 2 2 2 3 2 3 3 1 3 3 3 2 3 1 3 2 2 2 2 1 3 1 1 2 2 1
## [889] 1 2 1 1 1 1 2 2 3 2 1 2 1 1 3 1 1 1 3 1 1 2 1 2 1 3 3 1 1 3 1 3 1 3 3 2 2
## [926] 2 1 2 2 1 2 3 2 3 2 3 2 1 2 3 1 3 1 1 2 2 2 2 3 2 2 2 2 3
##
## Within cluster sum of squares by cluster:
## [1] 79773.84 81974.18 74101.65
##  (between_SS / total_SS =   8.0 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

Most of the data is a lot of numbers that are hard to comprehend. We do see a very short distance between the clusters. Generally the size of each cluster is:

```
ms_clusters_3$size
```

```
## [1] 340 318 296
```

For more characteristics, we'll merge the kmeans prediction back to the data:

```
ms_dat$cluster3 <- ms_clusters_3$cluster
```

When the first 5 individuals in the data are:

```
ms_dat[1:5, c("cluster3", "status", "gender", "race")]
```

```
##   cluster3 status gender                        race
## 1        3     MS      F Black or African American
## 2        1     MS      F                       White
## 3        1     HC      F                       White
## 4        2     HC      F                       White
## 5        1     HC      F                       White
```

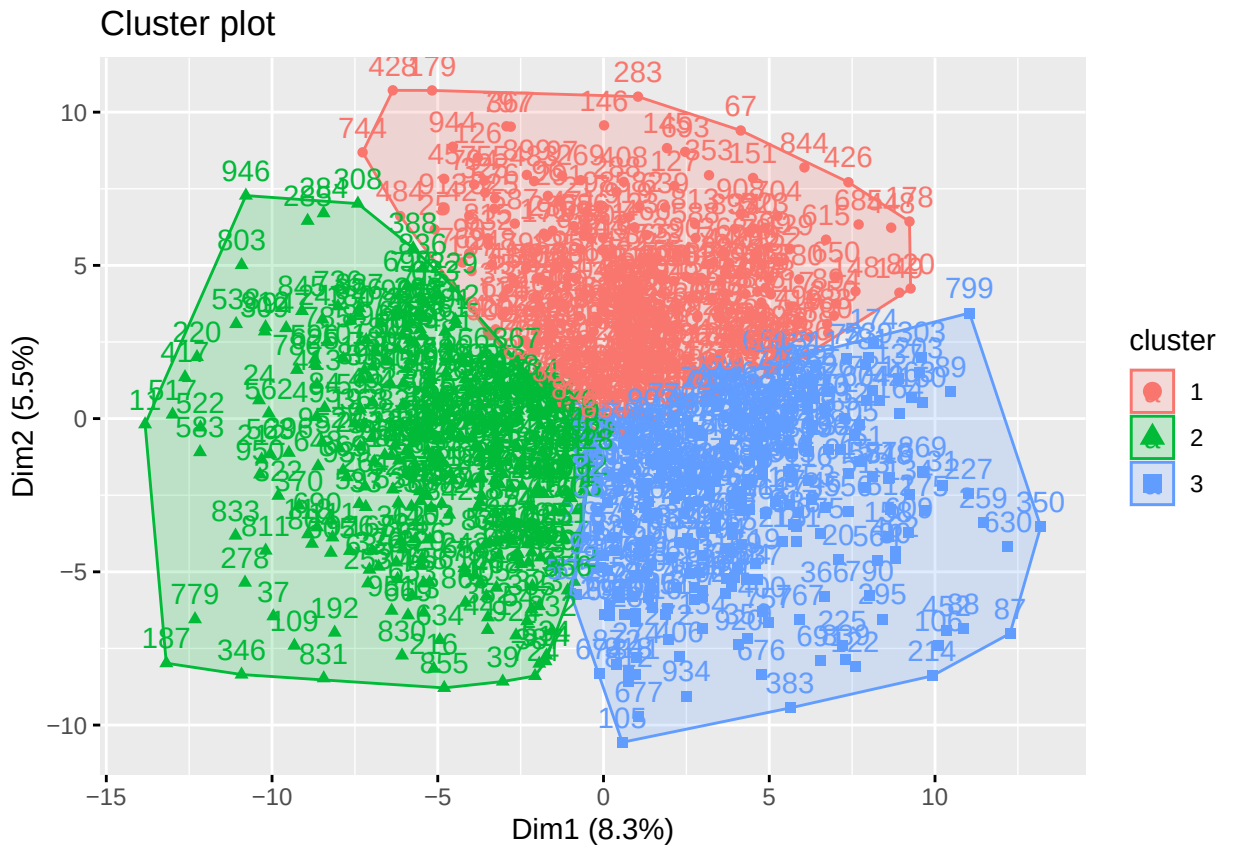
When the average age of each cluster:

```
aggregate(data = ms_dat, age ~ cluster3, mean)
```

```
##   cluster3      age
## 1        1 41.98529
## 2        2 41.20126
## 3        3 37.48311
```

Which is in agreement with ages of onset of MS, mainly females (data not shown). Let's map those clusters:

```
fviz_cluster(ms_clusters_3, data = ms_knum)
```



The clusters are quit close to one another due to the many metabolites. Deeper more thorough research is required to characterize each cluster (what was done in the paper with PCA) because we can have a lot of information just from the outliers. However, maybe this density is an indication that we need to choose a bigger k, like k=4. Let's check that:

```
set.seed(2345)
ms_clusters_4 <- kmeans(ms_dat_z, centers = 4, nstart = 25)
ms_clusters_4
```

```
## K-means clustering with 4 clusters of sizes 284, 252, 222, 196
##
## Cluster means:
##       X48762      X20675      X34400      X30460     X32350     X34395
## 1 -0.1161668  0.14721936 -0.02907742 -0.02600927 -0.2349970 -0.1251436
## 2 -0.2654714 -0.24611587 -0.26583672 -0.55290088 -0.3602086 -0.3071071
## 3  0.2440061  0.12261082  0.04037447  0.20967539  0.1096193  0.1667610
## 4  0.2332695 -0.03575868  0.33819239  0.51106999  0.6794707  0.3873001
##        X34389     X38395     X38168     X38276     X42374      X43761
## 1 -0.01232928 -0.1611431  0.3746270 -0.1452249  0.2621509 -0.15481659
## 2 -0.21289235 -0.1710305 -0.2014956 -0.3759066 -0.4451381 -0.11458241
## 3  0.10877990  0.2693014  0.1597710  0.4270810  0.2797314  0.04813665
## 4  0.16837373  0.1483644 -0.4647262  0.2100017 -0.1243695  0.31712420
##        X43343     X43266      X36746      X42489     X18281      X22036
## 1  0.14321489 -0.1375897 -0.04974979 -0.02048462 -0.1484660 -0.06885967
## 2 -0.28007865 -0.1886295 -0.37236180 -0.29658392 -0.1775624 -0.23763993
## 3  0.09777929  0.1622383  0.50363626  0.27766759 -0.1439939  0.23741771
## 4  0.04183566  0.2581285 -0.01960784  0.09650293  0.6065138  0.13640141
##       X35675     X45095     X46115     X32197       X31787      X32397
## 1  0.4265808 -0.2079722  0.4834419 -0.1629428  0.011836022  0.24297131
## 2 -0.3375225 -0.7059044 -0.4395380 -0.5409768 -0.252916501 -0.67528523
## 3 -0.3668551  0.6447749  0.3023663  0.5579864 -0.001548052  0.08073914
## 4  0.2313701  0.4786325 -0.4778533  0.2996373  0.309781609  0.42471395
##         X531        X542     X22053     X39600       X1549     X32457
## 1 -0.3457336  0.70086434  0.6284050 -0.2112062  0.03084473  0.6662179
## 2 -0.3584113 -0.34252010 -0.3911126 -0.2222017 -0.46825498 -0.3151319
## 3  0.3217102 -0.49682072 -0.5109442  0.2354669  0.44327162 -0.6775287
## 4  0.5973895 -0.01242963  0.1710356  0.3250190  0.05527578  0.2072385
##       X22001     X48448     X27672      X48763       X12017     X46165
## 1  0.4167795 -0.1685010 -0.1641750 -0.07932852  0.030364758 -0.1404589
## 2 -0.4188855 -0.3388696 -0.2650671 -0.34180950 -0.005338022 -0.3449410
## 3 -0.1679243  0.2763406  0.1393194  0.25619202 -0.155509766  0.3001679
## 4  0.1248620  0.3668460  0.4208862  0.26423810  0.139003869  0.3070315
##        X46548      X15677      X32445      X15749      X1558       X37181
## 1 -0.16012233 -0.07218803 -0.03952201  0.03475990 -0.1334141  0.03279829
## 2 -0.02750753 -0.28324114 -0.11707248 -0.20096742 -0.4298493 -0.29585915
## 3 -0.25048583  0.24630892 -0.11103396  0.06377847 -0.0383155  0.13342589
## 4  0.55109435  0.18978361  0.33355130  0.13578142  0.7893759  0.18174103
##        X36099      X48441      X35527       X1669      X46146     X36098
## 1 -0.05231065  0.11230970 -0.43423384 -0.33275348 -0.15789778 -0.1115046
## 2 -0.23597821 -0.23189007 -0.08572327 -0.19480212 -0.20366980 -0.1884557
## 3  0.09236811  0.04661285  0.24640647  0.56965232  0.04766505  0.1568186
## 4  0.27457660  0.08261373  0.46031856  0.08739443  0.43666386  0.2262469
##        X1418      X34424     X35136      X1494     X37186     X37190
## 1  0.1908151 -0.02251021  0.1841194 -0.1270222  0.4632749  0.4288388
```

```
## 2 -0.2995352 -0.29610823 -0.4697739 -0.3765262 -0.3691138 -0.5051724
## 3 -0.1928928  0.07333208  0.1596752  0.4427525  0.1899357  0.4065959
## 4  0.3271100  0.33026762  0.1563531  0.1666728 -0.4118322 -0.4324035
##         X37198      X37200       X37196     X43231     X36776       X35114
## 1  0.334995857  0.28555930  0.322548098 -0.3457041  0.2237163  0.045546272
## 2 -0.067905530  0.00289197  0.004805951 -0.1881482 -0.2872482 -0.389821265
## 3 -0.008608066 -0.04112971 -0.076529292  0.4672916 -0.1873682 -0.005972558
## 4 -0.388345098 -0.37090216 -0.386863554  0.2135438  0.2573819  0.441967986
##        X34390     X32198     X43258       X1126     X22132     X32827
## 1 -0.02851849  0.4812317  0.1527842 -0.42786029  0.1184080  0.2925921
## 2 -0.15310286 -0.5189831 -0.3437741 -0.15640778 -0.4100174 -0.2857565
## 3  0.02560237 -0.3584915 -0.1359574  0.70678264  0.4726530  0.2058108
## 4  0.20917064  0.3760155  0.3746068  0.02051702 -0.1797575 -0.2896710
##       X37209     X37202     X37203     X37211     X37210     X31591       X1638
## 1  0.5125656  0.4711699  0.5124340  0.5434726  0.4703245  0.5005576 -0.15312990
## 2 -0.3943778 -0.4238640 -0.4358131 -0.4157465 -0.3108980 -0.3902983  0.04560493
## 3  0.3861233  0.3565097  0.3927393  0.3501059  0.1303701  0.3168571  0.26847899
## 4 -0.6729836 -0.5415493 -0.6270127 -0.6494980 -0.4294287 -0.5823749 -0.14084635
##          X512      X12129       X3141     X32586     X43807        X2137
## 1 -0.22546640 -0.01335513 -0.06308478  0.5810595  0.3734624  0.392607519
## 2 -0.22442416 -0.36757010 -0.36299618 -0.3179986 -0.2553456 -0.294926260
## 3  0.58715318  0.36605394  0.33262291 -0.2344447 -0.1073754  0.002737054
## 4 -0.04979928  0.07732933  0.18137158 -0.1675435 -0.0912188 -0.192789509
##        X32412       X48782       X569       X1642      X32492      X15500
## 1  0.01511927 -0.006997574 -0.1056368  0.20396519 -0.07097165  0.06516161
## 2 -0.40007085 -0.464589598 -0.2089072 -0.27658021 -0.21515949 -0.43776998
## 3 -0.02831559 -0.186850357  0.1727708  0.01874972  0.28866535  0.21754119
## 4  0.52454103  0.819105453  0.2259712  0.03882481  0.05251241  0.22203059
##       X35320       X1563       X22842     X15506      X34396     X38637
## 1 -0.1834829 -0.07583396 -0.181696018 -0.2695682 -0.07221294  0.1667487
## 2 -0.3705742 -0.11052849  0.020679795 -0.3262707 -0.01830324 -0.1715388
## 3  0.3396003  0.21100930  0.214865693  0.4379316  0.28372612 -0.1744158
## 4  0.3576662  0.01298959 -0.006682772  0.3140651 -0.19319545  0.1764870
##       X38178       X1564       X2132       X1712       X1769      X27718
## 1  0.4352930  0.01282787  0.02019301 -0.02968449  0.19533977 -0.19825650
## 2 -0.3148385 -0.12173508 -0.39538270  0.03917847 -0.17085706  0.05560194
## 3 -0.6071922 -0.12914178 -0.02156152 -0.02275958  0.03503629  0.14069682
## 4  0.4617998  0.28420204  0.50351164  0.01841861 -0.10305393  0.05642073
##          X513      X18368     X22176     X33941     X32425     X36747
## 1  0.03366248  0.05145812 -0.1436674  0.4859329  0.5461863  0.1220437
## 2 -0.55890794 -0.20865592 -0.0565690 -0.3554136 -0.3043119 -0.5902241
## 3  0.30309916 -0.04918632  0.1081255 -0.6051203  0.3926998  0.3921605
## 4  0.32651349  0.24942118  0.1584340  0.4382448 -0.8449473  0.1378389
##           X1114      X36808      X5086      X32388      X48407      X39831
## 1 -0.2190950411 -0.02676277 -0.1759097  0.29448712 -0.24283352  0.12386092
## 2 -0.0006439054 -0.66071295 -0.3777661 -0.37098333 -0.16931139 -0.06692671
## 3  0.1323395803  0.15177246  0.4122672 -0.01658277  0.43816835  0.07797910
## 4  0.1683972907  0.71636124  0.2736331  0.06905527  0.07325417 -0.18174658
##       X33973      X37459      X20699     X42420      X15765     X47112
## 1  0.5172917  0.03972980 -0.22978177 -0.3205412  0.05891051  0.4391904
## 2 -0.4542045 -0.20909557 -0.18385576 -0.4232744 -0.36555071 -0.4070511
## 3  0.3511653  0.20528936  0.05442305  0.2475493 -0.14770387  0.1386512
## 4 -0.5633164 -0.02125214  0.50769264  0.7282801  0.55193088 -0.2700702
##          X1643      X44876     X36738       X2730      X18369     X44872
```

```
## 1  0.009387946 -0.15317488 -0.1370424  0.15985454 -0.28175408 -0.2487129
## 2 -0.410839415 -0.04721040 -0.3587689 -0.26027842 -0.49723609 -0.2845223
## 3  0.145959811  0.01342733  0.4233665  0.05542964  0.84988092  0.7331043
## 4  0.349297336  0.26743786  0.1803186  0.04023516  0.08493923 -0.1041585
##       X33422      X2734     X43829     X18280        X587      X15443
## 1 -0.3976050 -0.4096256 -0.2256782 -0.1346792 -0.28795333 -0.18272880
## 2 -0.5639957 -0.3845099 -0.3758149 -0.2816120 -0.15191014 -0.26136730
## 3  0.7736476  0.7622570  0.5644114  0.1519900  0.06666756 -0.06642661
## 4  0.4249845  0.2245363  0.1709115  0.3850681  0.53704033  0.67605228
##          X57         X53        X396       X1572      X15122      X37455
## 1 -0.2516278  0.08593881  0.01098326 -0.08679115  0.26324076 -0.15146878
## 2 -0.2369200 -0.23027473 -0.11836586 -0.16896401 -0.09593081 -0.06302533
## 3  0.4513806 -0.02495344  0.15719214  0.17111212 -0.50710188  0.28492397
## 4  0.1579574  0.19980751 -0.04177400  0.14918737  0.31627963 -0.02221227
##           X58      X32346      X18476      X18477      X42574       X32620
## 1  0.007786636 -0.40480635 -0.42179677 -0.37190051 -0.17467474  0.007655558
## 2 -0.183682968 -0.10417445 -0.06892164 -0.06387627 -0.02124598 -0.226366083
## 3  0.070911935  0.59553459  0.53491292  0.40806166  0.30143574  0.196072989
## 4  0.144562519  0.04596063  0.09391770  0.15881060 -0.06100571  0.057866689
##        X39379     X43802      X46957     X15753         X59       X22137
## 1 -0.23076516  0.1509379 -0.22526031 -0.1073768 -0.25977833 -0.080810491
## 2  0.06359321 -0.2313897 -0.30313299 -0.3554120 -0.08344472 -0.001814237
## 3  0.25975747  0.2044156 -0.03691269  0.1719204  0.54071945  0.362337823
## 4 -0.04160378 -0.1527368  0.75794927  0.4178189 -0.12874797 -0.290977702
##       X22138     X33009     X35322      X15716     X27513     X18349
## 1 -0.1951579 -0.1500969 -0.2277264  0.03736882 -0.1342022 -0.1103484
## 2 -0.2660988 -0.2748671 -0.1295700 -0.26162178 -0.2816280 -0.4354384
## 3  0.2059954  0.2288883  0.2334588  0.18321950  0.1601236  0.4594782
## 4  0.3915856  0.3116368  0.2321329  0.07470007  0.3751848  0.1993125
##        X32405     X33441       X1125      X44656     X34407        X1417
## 1 -0.02004868 -0.3333539 -0.32888161 -0.10815474 -0.2419118 -0.006037326
## 2 -0.08034596 -0.3835835 -0.43564673 -0.19888010 -0.4629846 -0.579487602
## 3  0.20292396  0.5583767  0.98905420  0.39415221  0.6803873  0.370046703
## 4 -0.09749037  0.3437548 -0.08359531 -0.03402071  0.1751485  0.334668919
##       X15140       X527     X34534        X60     X46223         X1301
## 1 -0.2003954 -0.1449947  0.5694657 -0.2431433  0.3439419 -0.2016446927
## 2 -0.3701890 -0.2144169 -0.5230514 -0.5068644 -0.5167195 -0.3786651167
## 3  0.2439986  0.2513857 -0.5118492  0.9779050 -0.1094763  0.6874894537
## 4  0.4899603  0.2010405  0.4270980 -0.1036347  0.2899876  0.0003471603
##         X1303     X20676      X1302      X44878      X18374      X46144
## 1 -0.09891404 -0.1863967 -0.2645474 -0.10324274 -0.44354144  0.03625028
## 2 -0.40440654 -0.2146690 -0.2600838 -0.38812184 -0.13647820 -0.17474845
## 3  0.33945404  0.1978559  0.7265545  0.02894067  0.77397714  0.01072960
## 4  0.27879204  0.3219860 -0.1052170  0.61583070 -0.05849311  0.15999775
##        X48429      X1496     X15745     X48182     X33952      X31536
## 1 -0.09889084 -0.1450243 -0.1358969  0.6353020  0.4717868 -0.09736571
## 2  0.12671753 -0.2702584 -0.4445004 -0.5010356 -0.6094758 -0.29290690
## 3 -0.16067158  0.1964473  0.1439386 -0.6235970 -0.2562334  0.24325150
## 4  0.16235344  0.3351057  0.6053799  0.4299681  0.3902259  0.24215596
##        X37432       X1585      X43488     X15720     X33943      X27710
## 1  0.05053361 -0.10735225  0.01502265 -0.3666203 -0.1852062  0.46328034
## 2 -0.21632021 -0.59532194 -0.41686963 -0.3482675 -0.1820607 -0.25498592
## 3  0.01798417  0.09243895  0.36995604  0.5184460  0.1044803 -0.38324660
## 4  0.18453399  0.81626388  0.09517588  0.3917784  0.3840981  0.09064071
```

```
##        X1587       X1589     X33950     X37496      X48187     X32390
## 1 -0.3505763 -0.07079930 -0.3408118 -0.1193187 -0.07589056 -0.3637935
## 2 -0.4012764 -0.19236921 -0.3831071 -0.2142795 -0.44839604 -0.2228319
## 3  0.6623960  0.07611486  0.4871485 -0.1674902  0.13208276  0.4766061
## 4  0.2736399  0.26370685  0.4346255  0.6381008  0.53686911  0.2737980
##        X1591      X43249       X2829      X47101      X37431     X42092
## 1 -0.1751583 -0.01865055 -0.01574493 -0.02963425 -0.04238069  0.4926097
## 2 -0.5586262 -0.45781746 -0.43799969 -0.40465952 -0.12824819 -0.2762704
## 3  0.3814406  0.33637462  0.04111123  0.01439746  0.19459617 -0.4429058
## 4  0.5399947  0.23465098  0.53939177  0.54690862  0.00588934  0.1430820
##        X40469      X35137       X1498        X594     X46111     X45413
## 1 -0.08255404 -0.06874388 -0.06400789  0.03227215 -0.1822971  0.1511248
## 2 -0.18707838 -0.41410995 -0.46572371 -0.10043729 -0.4474220 -0.5049525
## 3 -0.08157650 -0.22747764  0.33269147  0.04133768  0.2486309 -0.1816508
## 4  0.45254633  0.88968881  0.31470975  0.03555073  0.5577891  0.6359953
##       X33936     X38102     X35160      X1493       X1505      X20694
## 1  0.4522980  0.3847654  0.4677109 -0.2033000 -0.03845298  0.05061708
## 2 -0.3169294 -0.3190150 -0.6008477 -0.5419016 -0.32186673  0.01964973
## 3 -0.6159033 -0.3839951 -0.2535490  0.6045022  0.31542350 -0.14871044
## 4  0.4497148  0.2875782  0.3819979  0.3066170  0.11228083  0.06983029
##       X36103      X48841     X44681       X1508     X18254      X32553
## 1 -0.1207868 -0.14192581  0.4400602  0.05230476  0.0163937 -0.02091386
## 2 -0.1467667 -0.06866445 -0.6062454 -0.31794960 -0.3037938 -0.21013081
## 3 -0.1489903 -0.23808582 -0.1714290 -0.04784114  0.1374811  0.09395517
## 4  0.5324720  0.56359909  0.3359897  0.38719122  0.2111195  0.19405333
##       X35126        X64      X22130        X566       X1444      X33935
## 1 -0.1969147 -0.3969046 -0.17941521 -0.22277848 -0.05833205 -0.03109429
## 2 -0.1892474 -0.3946510 -0.38067783 -0.12253752 -0.06430303  0.01082033
## 3 -0.1645127  0.7709613  0.61145373  0.50187996  0.25659547  0.11325088
## 4  0.7149793  0.2092834  0.05684696 -0.08810616 -0.12343637 -0.09713082
##       X32619      X40708     X32562     X38170     X35127      X1898     X40703
## 1  0.5290429  0.28309824  0.4627053  0.3878416 -0.1256276 -0.4667312 -0.4094237
## 2 -0.3706343 -0.01598769 -0.3810796 -0.2889726 -0.2797039 -0.2491740 -0.3363623
## 3  0.3431850 -0.06695749  0.3222523  0.3675477  0.1022906  0.8448158  0.7677911
## 4 -0.6787519 -0.31380837 -0.5454912 -0.6067423  0.4257914  0.0397674  0.1560714
##       X32452      X33442      X31555     X46225      X18335     X15772
## 1 -0.3598480  0.06238463 -0.08041195  0.1049555 -0.04752798 -0.3035038
## 2 -0.4499370 -0.54083837 -0.16687633 -0.3785733 -0.32337975 -0.1792043
## 3  0.7252587 -0.16718749 -0.05123086  0.2056673  0.15362404  0.1783679
## 4  0.2784364  0.79433499  0.38909734  0.1017089  0.31063830  0.4681474
##        X43239        X39592       X1515       X1648     X17747      X34445
## 1  0.01914388  1.369339e-01 -0.21630698 -0.04357639 -0.1192430 -0.04752148
## 2 -0.13198426 -1.423171e-01 -0.08104502 -0.14563737 -0.1592228 -0.25118873
## 3  0.11047684 -3.695991e-05 -0.13587587  0.43737974  0.1484017  0.14266218
## 4  0.01682302 -1.539341e-02  0.57152537 -0.24501016  0.2094081  0.23022785
##         X34384     X34409       X1437     X37058     X46960       X20693
## 1 -0.004274071  0.3461397 -0.04093638 -0.3692801 -0.2488889  0.042504202
## 2 -0.145011886 -0.6216008 -0.32290908 -0.2247759 -0.3061108 -0.023153680
## 3  0.094428261  0.0311756  0.05843854  0.2716134  0.2325063 -0.009768848
## 4  0.085682436  0.2623405  0.40829420  0.5164331  0.4908570 -0.020753988
##        X18494     X18497     X32807      X12261      X36850      X18392
## 1 -0.38971235 -0.2652152  0.1236439 -0.42052407 -0.01895119 -0.03619948
## 2 -0.06971823 -0.1521512 -0.1353209 -0.05378205 -0.20573299 -0.10835280
## 3  0.50755782  0.3387396  0.1901793  0.44094649  0.20969668  0.13055280
```

```
## 4  0.07943605  0.1962400 -0.2205806  0.17903975  0.05446015  0.04389201
##         X18394       X27738      X1284      X36095       X2761     X35428
## 1 -0.009839919 -0.006694567 -0.2129719 -0.03116615  0.01305924  0.0230571
## 2 -0.243155237 -0.027783707 -0.0993921 -0.10798239  0.07488644 -0.5423620
## 3  0.087989067 -0.084908778  0.4943544  0.19076404 -0.15484799  0.3688794
## 4  0.227224918  0.141594388 -0.1235503 -0.03207585  0.06018390  0.2461010
##        X32306        X607     X32401        X54      X37097      X48408
## 1 -0.20114212  0.05915139 -0.1244386 -0.2312888 -0.03347531 -0.29418235
## 2 -0.07738696  0.04993400 -0.3301517 -0.2056049 -0.11551441 -0.02225031
## 3  0.20740277 -0.10303054  0.1824230  0.7619096  0.07064214  0.24390051
## 4  0.15603297 -0.03321215  0.3981678 -0.2634973  0.11701054  0.17861711
##        X1299      X1604      X1670       X606       X1605       X1649
## 1 -0.4650772  0.1176119 -0.2338481  0.3159344 -0.05859991 -0.30421171
## 2 -0.3123004 -0.5812668 -0.4146137 -0.3272077  0.03607520 -0.46334025
## 3  0.8602162  0.2274771  0.3087631 -0.1934836 -0.06137257  0.93185335
## 4  0.1010899  0.3192730  0.5221945  0.1820629  0.10804151 -0.01894681
##         X1567       X3147     X15679
## 1 -0.11076131  0.05533400 -0.1356583
## 2 -0.41611493 -0.17694171 -0.2579361
## 3  0.02209559  0.05176352  0.5670312
## 4  0.67046915  0.08868855 -0.1140515
##
## Clustering vector:
##   [1] 2 1 1 3 4 2 1 1 4 2 3 4 3 1 1 2 1 1 2 2 1 1 2 3 4 4 2 3 4 2 2 1 3 2 1 4 3
##  [38] 2 3 1 2 2 2 1 4 2 1 2 1 4 1 3 1 2 4 4 1 1 1 1 2 3 2 2 2 1 1 3 3 3 2 2 4 2
##  [75] 4 1 2 1 1 2 2 2 1 4 4 4 2 2 2 4 4 1 1 2 4 4 4 1 3 2 3 1 1 1 2 2 2 3 3 3 3
## [112] 3 4 4 4 2 2 3 2 3 3 2 1 3 1 1 1 4 1 1 2 2 2 2 1 3 2 2 4 2 2 1 3 1 1 1 1 1
## [149] 1 2 1 1 1 2 1 1 3 2 2 2 2 3 4 2 2 2 3 3 2 1 1 4 2 1 2 4 4 1 4 4 4 4 2 1 3
## [186] 2 3 4 1 1 1 3 4 4 2 4 4 4 2 1 4 2 2 2 2 3 3 4 1 4 4 3 2 2 2 3 1 2 3 3 1 1
## [223] 2 4 2 2 2 2 4 4 1 3 2 4 4 3 2 3 3 1 1 3 4 3 1 4 2 4 1 1 3 2 3 3 3 3 2 3 2
## [260] 3 1 3 2 3 1 1 1 1 1 2 1 2 2 3 1 1 1 3 2 2 1 1 1 4 4 3 3 1 1 1 4 2 2 4 2 4
## [297] 4 1 1 4 4 3 2 2 3 4 2 4 4 3 2 4 1 1 2 1 1 3 2 1 1 3 3 4 4 1 4 4 4 3 4 2 4
## [334] 1 2 3 2 1 1 1 2 3 1 1 4 3 2 1 3 2 3 3 1 3 2 4 1 1 2 1 3 3 3 2 1 2 1 1 4 3
## [371] 3 2 2 1 1 3 1 1 2 1 1 2 2 4 1 1 2 4 3 1 3 3 2 1 4 1 3 1 2 2 1 2 1 1 3 2 1
## [408] 1 4 1 2 4 4 1 3 4 4 1 1 3 3 3 1 4 4 1 4 4 3 4 4 1 2 4 4 2 1 3 3 1 3 1 2 1
## [445] 2 2 1 1 3 4 4 2 3 1 1 1 2 1 4 4 4 2 2 2 4 1 3 3 1 2 1 1 1 4 2 3 3 1 2 2 3
## [482] 4 4 4 1 3 2 3 2 1 1 4 2 3 3 1 3 2 2 2 4 3 4 3 3 3 3 3 1 3 2 2 2 3 1 1 3 3
## [519] 2 1 3 3 1 1 1 2 2 4 2 2 2 2 2 3 3 3 3 3 2 1 1 3 1 4 4 4 4 4 3 1 2 2 2 2 1
## [556] 2 2 4 1 1 3 3 3 2 2 3 1 1 4 4 3 3 3 3 3 4 3 1 2 1 1 1 3 1 3 4 4 1 4 3 4 4
## [593] 4 4 4 3 4 4 1 1 1 1 4 2 3 1 2 2 2 1 2 1 3 3 1 4 4 4 3 3 1 4 1 4 4 4 4 4 4
## [630] 2 3 1 3 3 3 1 1 1 1 2 3 1 2 3 1 2 4 3 1 1 3 3 3 2 4 3 4 3 2 3 2 4 1 2 3 1
## [667] 3 1 3 1 3 3 2 2 2 2 2 1 1 1 2 4 1 2 1 3 3 3 1 3 2 4 1 1 4 4 2 2 4 1 2 3 2
## [704] 1 1 1 1 1 1 1 2 1 2 1 4 1 1 4 2 1 2 1 1 3 1 3 2 2 4 1 1 1 1 4 3 4 3 4 1 1
## [741] 3 2 2 4 2 3 1 2 2 1 2 2 2 2 4 4 2 4 2 3 2 1 2 3 3 4 2 4 4 4 4 2 1 1 1 1 2
## [778] 4 3 4 2 4 1 3 4 4 1 3 2 2 4 2 3 4 4 2 4 1 1 2 2 2 4 3 2 1 2 1 1 4 3 3 1 4
## [815] 1 2 2 4 1 2 3 4 1 1 2 4 2 2 4 3 3 3 3 3 3 4 1 2 2 1 3 3 3 1 4 3 2 1 3 4 2
## [852] 4 3 1 3 3 3 1 3 2 3 2 3 3 3 2 4 2 2 1 2 3 2 3 2 1 2 4 3 3 4 1 2 4 1 4 4 4
## [889] 2 3 1 1 1 1 3 4 2 4 1 4 1 1 1 4 1 1 2 1 1 4 1 4 4 2 2 4 1 2 1 2 1 2 2 3 3
## [926] 3 1 1 4 1 4 2 3 2 3 1 3 1 3 1 1 3 1 1 3 4 3 3 2 3 3 3 3 2
##
## Within cluster sum of squares by cluster:
## [1] 63763.93 62817.70 54257.93 49893.23
##  (between_SS / total_SS =  10.0 %)
##
```

```
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

This time the distance between the clusters is a bit larger, when the size of each cluster is:

```
ms_clusters_4$size
```

```
## [1] 284 252 222 196
```

For more characteristics, we'll merge the kmeans prediction back to the data:

```
ms_dat$cluster4 <- ms_clusters_4$cluster
```

When the first 5 individuals in the data are:

```
ms_dat[1:5, c("cluster4", "status", "gender", "race")]
```

```
##   cluster4 status gender                     race
## 1        2     MS      F Black or African American
## 2        1     MS      F                    White
## 3        1     HC      F                    White
## 4        3     HC      F                    White
## 5        4     HC      F                    White
```

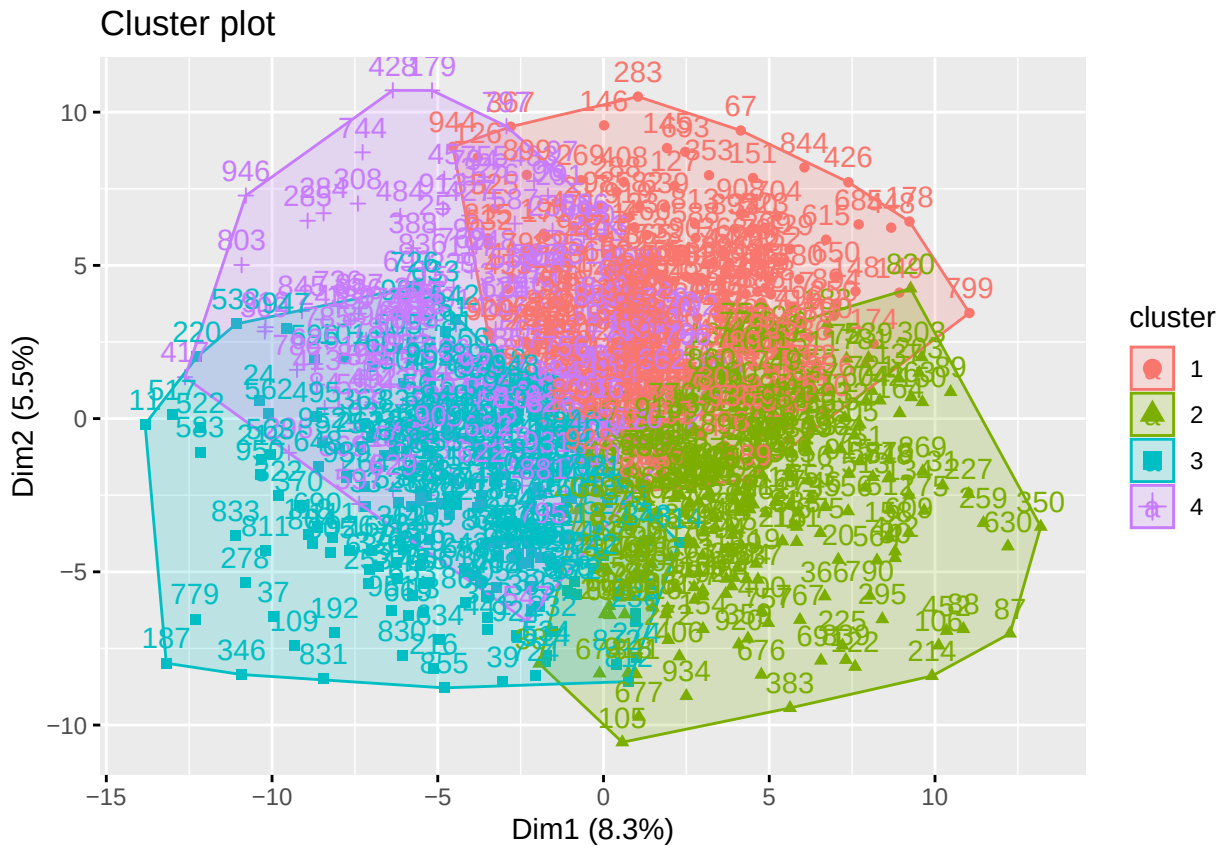
When the average age of each cluster:

```
aggregate(data = ms_dat, age ~ cluster4, mean)
```

```
##   cluster4      age
## 1        1 37.01408
## 2        2 38.46825
## 3        3 37.17117
## 4        4 51.09184
```

Here the range of ages is broader 37-51, mainly females (data not shown). Let's map those clusters:

```
fviz_cluster(ms_clusters_4, data = ms_knum)
```



However, with a greater K the overlap is higher than the previous K, which is not good for classification because overlapping of clusters can lead to higher erroneous classification. Hence, with the kmeans the best k for our data is 3. The information within each cluster requires further investigation that are broader than the scope of this report. Moreover, dimensional reduction may help as was done with PCA as shown in Fitzgerald et al. 2021 or other methods as we'll exemplify in the section after the next.

The Random Forest model

One of the most stringent models for this kind of classification is the random forest. This algorithm chooses the best decision tree out of many possible decision trees. This model requires the pre-processing steps of setting seed, splitting the data, and creating training and test sets. Moreover, the model requires choosing one factor for testing correlation. Perhaps using the random forest on the gender factor: male or female, will yield a better model.

```
# Random forest requires setting a seed
set.seed(111)
ms_split_gen <- initial_split(ms_dat %>% dplyr::select(-c(record_id, batch)),
                              strata = gender)
ms_train_gen <- training(ms_split_gen)
ms_test_gen  <- testing(ms_split_gen)

# Checking the results of the split
nrow(ms_train_gen)
```

```
## [1] 715
```

```
nrow(ms_train_gen)/nrow(ms_split_gen)
```

```
## analysis
##        1
```

```
# Training set proportions by gender
ms_train_gen %>%
  count(gender) %>%
  mutate(prop = n/sum(n))
```

```
##   gender   n      prop
## 1      F 519 0.7258741
## 2      M 196 0.2741259
```

```
# Testing set proportions by gender
ms_test_gen %>%
  count(gender) %>%
  mutate(prop = n/sum(n))
```

```
##   gender   n      prop
## 1      F 173 0.7238494
## 2      M  66 0.2761506
```

Seeing that the proportions of the training and test sets are more or less similar to the data set (reminder: females are about 70% of the participants in the research), the next step will be to define the number of trees in the random forest model, 1000, and the engine "ranger":

```
# Creating random forest model
rf_mod_gen <-
  rand_forest(trees = 1000) %>%
  set_engine("ranger") %>%
  set_mode("classification")
```

With a defined model, we can start the training step with new seed for reproducible results:

```
# Training the model
set.seed(222)
rf_fit_gen <-
  rf_mod_gen %>%
  fit(gender ~ ., data = ms_train_gen)
rf_fit_gen
```

```
## parsnip model object
##
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, num.trees = ~1000,      num.threads = 1, verbose = F
##
## Type:                             Probability estimation
```

```
## Number of trees:                  1000
## Sample size:                      715
## Number of independent variables:  274
## Mtry:                             16
## Target node size:                 10
## Variable importance mode:         none
## Splitrule:                        gini
## OOB prediction error (Brier s.):  0.1281926
```

According to ranger, the requirements and execution of our model were met with a low rate of prediction error, 0.12. The next step will be to understand the trade-off between the sensitivity and specificity and the classification accuracy using the Receiver Operating Characteristic (ROC).

```
# Calculating our model's prediction abilities
rf_training_gen_pred <-
  predict(rf_fit_gen, ms_train_gen) %>%
  bind_cols(predict(rf_fit_gen, ms_train_gen, type = "prob")) %>%
  bind_cols(ms_train_gen %>% dplyr::select(gender))

# ROC calculation
rf_training_gen_pred %>%
  roc_auc(truth = gender, .pred_F)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary             1
```

```
# Accuracy calculation
rf_training_gen_pred %>%
  accuracy(truth = gender, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary             1
```

In contrast to what we observed in the random forest of status of health, in this model the accuracy is (1!) and the ROC estimate (1!) are very high, and may indicate on an over fitting. Is this maintained in the test set as well?

```
rf_testing_gen_pred <-
  predict(rf_fit_gen, ms_test_gen) %>%
  bind_cols(predict(rf_fit_gen, ms_test_gen, type = "prob")) %>%
  bind_cols(ms_test_gen %>% dplyr::select(gender))

# ROC calculation
rf_testing_gen_pred %>%
  roc_auc(truth = gender, .pred_F)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.921
```

```r
# Accuracy calculation
rf_testing_gen_pred %>%
  accuracy(truth = gender, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.837
```

In the test set the ROC (0.91) as well as the accuracy (0.83) are less in comparison to the training test, but great and more realistic. Is it because of the analysis or a real stable model? re-sampling with 10-fold cross-validation can help in that:

```r
# Creating 10 mini data sets from the training data
set.seed(333)
folds <- vfold_cv(ms_train_gen, v = 10)
folds
```

```
## #  10-fold cross-validation
## # A tibble: 10 x 2
##    splits           id
##    <list>           <chr>
##  1 <split [643/72]> Fold01
##  2 <split [643/72]> Fold02
##  3 <split [643/72]> Fold03
##  4 <split [643/72]> Fold04
##  5 <split [643/72]> Fold05
##  6 <split [644/71]> Fold06
##  7 <split [644/71]> Fold07
##  8 <split [644/71]> Fold08
##  9 <split [644/71]> Fold09
## 10 <split [644/71]> Fold10
```

```r
# Setting the random forest workflow that bundles together the model specifications and actual modeling
rf_gen_wf <-
  workflow() %>%
  add_model(rf_mod_gen) %>%
  add_formula(gender ~ .)

# add folds to workflow and train model
set.seed(444)
rf_fit_gen <-
  rf_gen_wf %>%
  fit_resamples(folds)

rf_fit_gen
```

```
## # Resampling results
## # 10-fold cross-validation
## # A tibble: 10 x 4
##    splits           id     .metrics         .notes
##    <list>           <chr>  <list>           <list>
```

```
##  1 <split [643/72]> Fold01 <tibble [2 x 4]> <tibble [0 x 3]>
##  2 <split [643/72]> Fold02 <tibble [2 x 4]> <tibble [0 x 3]>
##  3 <split [643/72]> Fold03 <tibble [2 x 4]> <tibble [0 x 3]>
##  4 <split [643/72]> Fold04 <tibble [2 x 4]> <tibble [0 x 3]>
##  5 <split [643/72]> Fold05 <tibble [2 x 4]> <tibble [0 x 3]>
##  6 <split [644/71]> Fold06 <tibble [2 x 4]> <tibble [0 x 3]>
##  7 <split [644/71]> Fold07 <tibble [2 x 4]> <tibble [0 x 3]>
##  8 <split [644/71]> Fold08 <tibble [2 x 4]> <tibble [0 x 3]>
##  9 <split [644/71]> Fold09 <tibble [2 x 4]> <tibble [0 x 3]>
## 10 <split [644/71]> Fold10 <tibble [2 x 4]> <tibble [0 x 3]>
```

```
collect_metrics(rf_fit_gen)
```

```
## # A tibble: 2 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy binary     0.843    10  0.0146 Preprocessor1_Model1
## 2 roc_auc  binary     0.900    10  0.0116 Preprocessor1_Model1
```

The re-sampling maintained the values of the ROC (0.84) and the accuracy (0.9) estimates. Or generally the estimates are:

```
# ROC calculation
rf_testing_gen_pred %>%
  roc_auc(truth = gender, .pred_F)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.921
```

```
# Accuracy calculation
rf_testing_gen_pred %>%
  accuracy(truth = gender, .pred_class)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.837
```

The predictions of the estimates indicate that this model does classifies very well (accuracy 0f 0.83 and ROC of 0.91), without over fitting. will tuning improve the model?

```
# Defining the parameters for the decision tree
tune_spec_gen <-
  decision_tree(
    cost_complexity = tune(),
    tree_depth = tune()
  ) %>%
  set_engine("rpart") %>%
  set_mode("classification")

tune_spec_gen
```

```
## Decision Tree Model Specification (classification)
##
## Main Arguments:
##   cost_complexity = tune()
##   tree_depth = tune()
##
## Computational engine: rpart
```

```
# Creating 5 'datasets' with different combinations
tree_grid <- grid_regular(cost_complexity(),
                          tree_depth(),
                          levels = 5)

tree_grid
```

```
## # A tibble: 25 x 2
##    cost_complexity tree_depth
##              <dbl>      <int>
##  1    0.0000000001          1
##  2    0.0000000178          1
##  3    0.00000316            1
##  4    0.000562              1
##  5    0.1                   1
##  6    0.0000000001          4
##  7    0.0000000178          4
##  8    0.00000316            4
##  9    0.000562              4
## 10    0.1                   4
## # ... with 15 more rows
```

```
tree_grid %>%
  count(tree_depth)
```

```
## # A tibble: 5 x 2
##   tree_depth     n
##        <int> <int>
## 1          1     5
## 2          4     5
## 3          8     5
## 4         11     5
## 5         15     5
```

```
set.seed(555)
ms_folds_gen <- vfold_cv(ms_train_gen) # create the actual cross-validation folds
```

Now let's add the grid to this model:

```
set.seed(666)
# Defining workflow
tree_gen_wf <- workflow() %>%
  add_model(tune_spec_gen) %>%
  add_formula(gender ~ .)
```

```
# Updating and training workflow with grids
tree_res_gen <-
  tree_gen_wf %>%
  tune_grid(
    resamples = ms_folds_gen,
    grid = tree_grid
    )

tree_res_gen
```

```
## # Tuning results
## # 10-fold cross-validation
## # A tibble: 10 x 4
##    splits           id     .metrics          .notes
##    <list>           <chr>  <list>            <list>
##  1 <split [643/72]> Fold01 <tibble [50 x 6]> <tibble [0 x 3]>
##  2 <split [643/72]> Fold02 <tibble [50 x 6]> <tibble [0 x 3]>
##  3 <split [643/72]> Fold03 <tibble [50 x 6]> <tibble [0 x 3]>
##  4 <split [643/72]> Fold04 <tibble [50 x 6]> <tibble [0 x 3]>
##  5 <split [643/72]> Fold05 <tibble [50 x 6]> <tibble [0 x 3]>
##  6 <split [644/71]> Fold06 <tibble [50 x 6]> <tibble [0 x 3]>
##  7 <split [644/71]> Fold07 <tibble [50 x 6]> <tibble [0 x 3]>
##  8 <split [644/71]> Fold08 <tibble [50 x 6]> <tibble [0 x 3]>
##  9 <split [644/71]> Fold09 <tibble [50 x 6]> <tibble [0 x 3]>
## 10 <split [644/71]> Fold10 <tibble [50 x 6]> <tibble [0 x 3]>
```

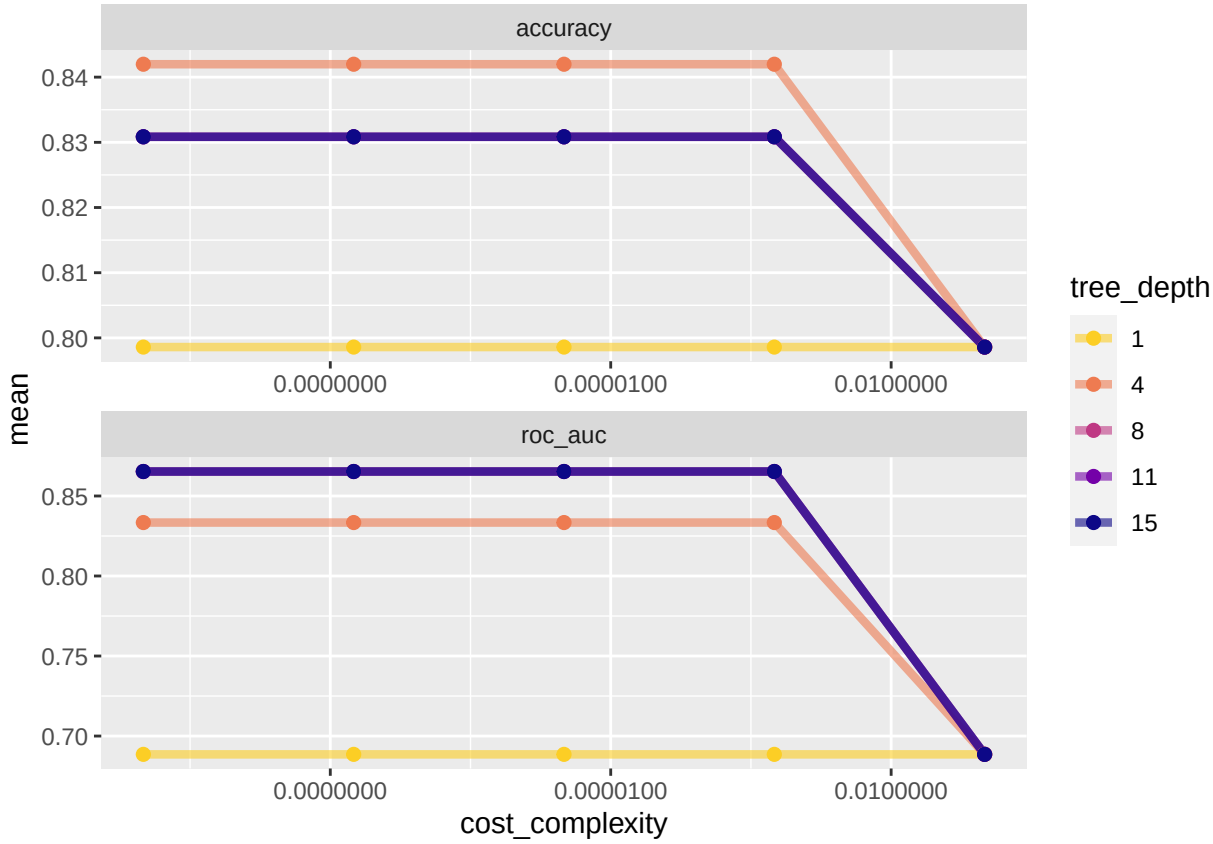
```
tree_res_gen %>%
  collect_metrics()
```

```
## # A tibble: 50 x 8
##    cost_complexity tree_depth .metric  .estimator  mean     n std_err .config
##              <dbl>      <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
##  1    0.0000000001          1 accuracy binary     0.799    10 0.00980 Preproces~
##  2    0.0000000001          1 roc_auc  binary     0.689    10 0.0187  Preproces~
##  3    0.0000000178          1 accuracy binary     0.799    10 0.00980 Preproces~
##  4    0.0000000178          1 roc_auc  binary     0.689    10 0.0187  Preproces~
##  5    0.00000316            1 accuracy binary     0.799    10 0.00980 Preproces~
##  6    0.00000316            1 roc_auc  binary     0.689    10 0.0187  Preproces~
##  7    0.000562              1 accuracy binary     0.799    10 0.00980 Preproces~
##  8    0.000562              1 roc_auc  binary     0.689    10 0.0187  Preproces~
##  9    0.1                   1 accuracy binary     0.799    10 0.00980 Preproces~
## 10    0.1                   1 roc_auc  binary     0.689    10 0.0187  Preproces~
## # ... with 40 more rows
```

The Tuning has worsen our model: the ROC estimate (0.68) and the accuracy (0.79) decreased. In order to understand what happened in the process that led to these estimates, we'll plot the performance:

```
# plot performance
tree_res_gen %>%
  collect_metrics() %>%
  mutate(tree_depth = factor(tree_depth)) %>%
```

```
  ggplot(aes(cost_complexity, mean, color = tree_depth)) +
  geom_line(size = 1.5, alpha = 0.6) +
  geom_point(size = 2) +
  facet_wrap(~ .metric, scales = "free", nrow = 2) +
  scale_x_log10(labels = scales::label_number()) +
  scale_color_viridis_d(option = "plasma", begin = .9, end = 0)
```



The accuracy was the best when the decision tree was of depth 4, while the cost complexity is higher from tree depth 8. Let's use the show_best() function to see the top 5 candidate models by default:

```
# best tree
tree_res_gen %>%
  show_best("roc_auc")
```

```
## # A tibble: 5 x 8
##   cost_complexity tree_depth .metric .estimator  mean     n std_err .config
##             <dbl>      <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    0.0000000001          8 roc_auc binary     0.865    10  0.0194 Preprocesso~
## 2    0.0000000178          8 roc_auc binary     0.865    10  0.0194 Preprocesso~
## 3    0.00000316            8 roc_auc binary     0.865    10  0.0194 Preprocesso~
## 4    0.000562              8 roc_auc binary     0.865    10  0.0194 Preprocesso~
## 5    0.0000000001         11 roc_auc binary     0.865    10  0.0194 Preprocesso~
```

The first 4 best models are of depth 8 and one of 11. But which is the best? Workflow can determine that as follows:

```
best_tree_gen <- tree_res_gen %>%
  select_best("roc_auc") # Selecting the best tree

best_tree_gen
```

```
## # A tibble: 1 x 3
##   cost_complexity tree_depth .config
##             <dbl>      <int> <chr>
## 1    0.0000000001          8 Preprocessor1_Model11
```

```
# Finalize workflow
final_wf_gen <-
  tree_gen_wf %>%
  finalize_workflow(best_tree_gen)

final_wf_gen
```

```
## == Workflow ====================================================================
## Preprocessor: Formula
## Model: decision_tree()
##
## -- Preprocessor ----------------------------------------------------------------
## gender ~ .
##
## -- Model -----------------------------------------------------------------------
## Decision Tree Model Specification (classification)
##
## Main Arguments:
##   cost_complexity = 1e-10
##   tree_depth = 8
##
## Computational engine: rpart
```

Now that we found the best tree, let's look who is this tree:

```
final_tree_gen <-
  final_wf_gen %>%
  fit(data = ms_train_gen)

final_tree_gen
```

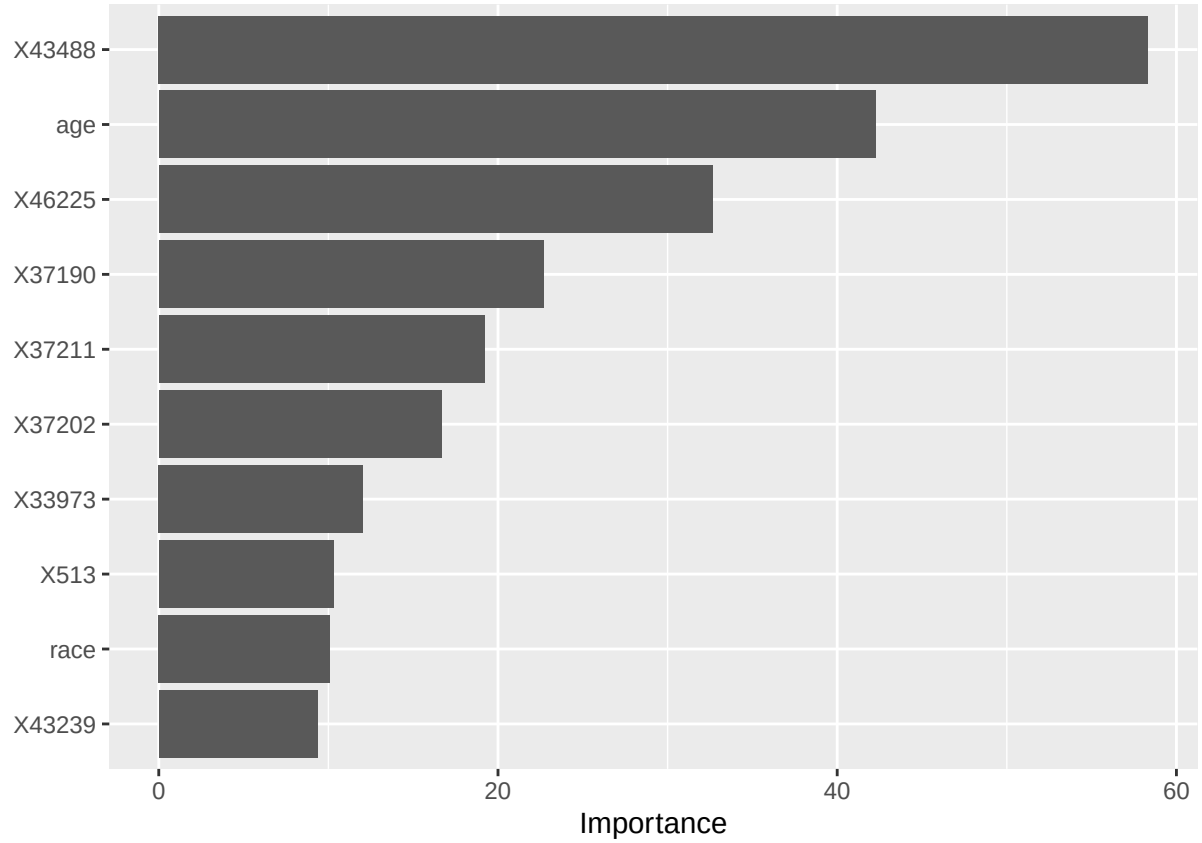
```
## == Workflow [trained] ==========================================================
## Preprocessor: Formula
## Model: decision_tree()
##
## -- Preprocessor ----------------------------------------------------------------
## gender ~ .
##
## -- Model -----------------------------------------------------------------------
## n= 715
##
## node), split, n, loss, yval, (yprob)
```

```
##       * denotes terminal node
##
##  1) root 715 196 F (0.72587413 0.27412587)
##    2) X43488< 14.80133 588 106 F (0.81972789 0.18027211)
##      4) age>=20.5 505  63 F (0.87524752 0.12475248)
##        8) X46225< 18.70072 438  27 F (0.93835616 0.06164384)
##         16) X37190< 15.6194 398  11 F (0.97236181 0.02763819) *
##         17) X37190>=15.6194 40  16 F (0.60000000 0.40000000)
##           34) X40708>=13.01319 22   2 F (0.90909091 0.09090909) *
##           35) X40708< 13.01319 18   4 M (0.22222222 0.77777778) *
##        9) X46225>=18.70072 67  31 M (0.46268657 0.53731343)
##         18) X37190< 14.74266 34   8 F (0.76470588 0.23529412)
##           36) X18392>=17.53092 24   1 F (0.95833333 0.04166667) *
##           37) X18392< 17.53092 10   3 M (0.30000000 0.70000000) *
##         19) X37190>=14.74266 33   5 M (0.15151515 0.84848485)
##           38) X35527>=14.44616 7   3 F (0.57142857 0.42857143) *
##           39) X35527< 14.44616 26   1 M (0.03846154 0.96153846) *
##      5) age< 20.5 83  40 M (0.48192771 0.51807229)
##       10) X43239>=13.39478 58  21 F (0.63793103 0.36206897)
##         20) X35127>=16.05945 49  13 F (0.73469388 0.26530612)
##           40) X22132>=13.33658 41   7 F (0.82926829 0.17073171)
##             80) X57< 20.25289 32   2 F (0.93750000 0.06250000) *
##             81) X57>=20.25289 9   4 M (0.44444444 0.55555556) *
##           41) X22132< 13.33658 8   2 M (0.25000000 0.75000000) *
##         21) X35127< 16.05945 9   1 M (0.11111111 0.88888889) *
##       11) X43239< 13.39478 25   3 M (0.12000000 0.88000000) *
##    3) X43488>=14.80133 127  37 M (0.29133858 0.70866142)
##      6) age< 26.5 28   5 F (0.82142857 0.17857143)
##       12) X34407< 15.66934 21   1 F (0.95238095 0.04761905) *
##       13) X34407>=15.66934 7   3 M (0.42857143 0.57142857) *
##      7) age>=26.5 99  14 M (0.14141414 0.85858586)
##       14) X46225< 18.23089 24  11 M (0.45833333 0.54166667)
##         28) X37455< 15.68686 11   1 F (0.90909091 0.09090909) *
##         29) X37455>=15.68686 13   1 M (0.07692308 0.92307692) *
##       15) X46225>=18.23089 75   3 M (0.04000000 0.96000000) *
```

When the feature that impacted on the gender classification the most is:

```
final_tree_gen %>%
  pull_workflow_fit() %>%
  vip()
```

```
## Warning: ‘pull_workflow_fit()‘ was deprecated in workflows 0.2.3.
## Please use ‘extract_fit_parsnip()‘ instead.
```



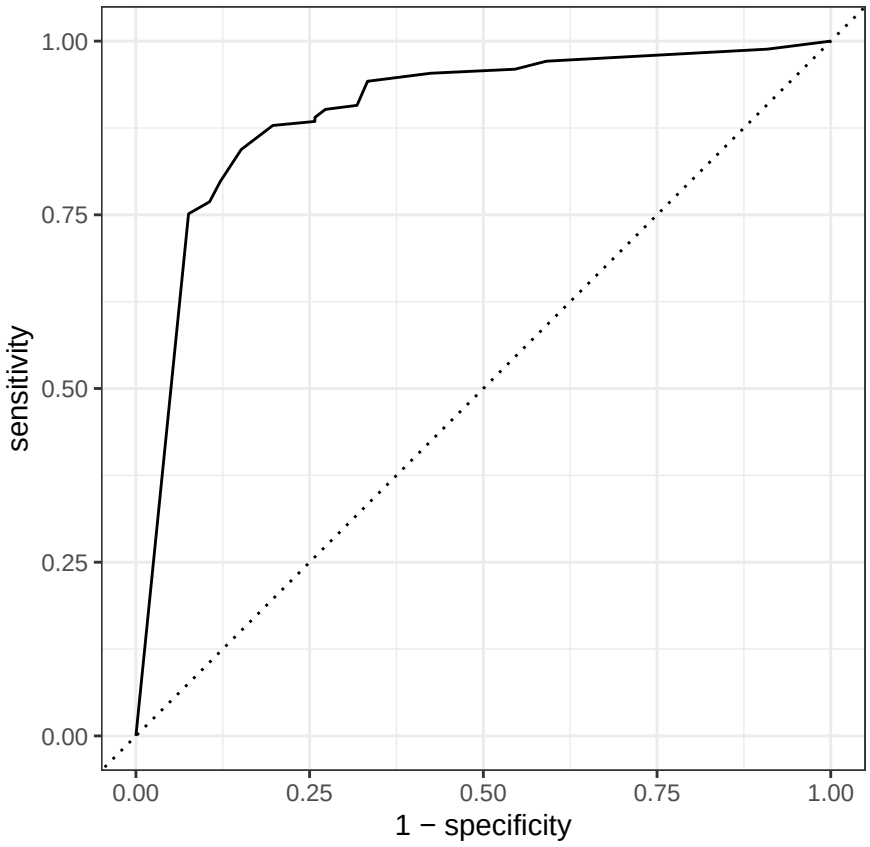
N-acetylcarnosine, used as eye drops in preventing cataract, and the age, which makes sense because the onset of the disease is around 20 and 40 years of age. Metabolom wise, the metabolites that affected the classification the most after those were (in descending order): Pyroglutamine amino acid; 3-hydroxylated C19 steroid hormones that are known to favor the development of masculine characteristics; two types of sterol lipids with a sulfate group (37202 and 37211); Epiandrosterone sulfate, a sulfated steroid; and creatinine, a product of breaking of creatine phosphate from muscle and protein metabolism. The last two influence is the race and the derivative of the amino acid cysteine S-allylcysteine. Interestingly, the status and gender classification models don't classify by the same parameters which makes it harder to decide which has greater influence. However, the significance of the latter seems higher. Let's check the trade-off between the sensitivity and specificity in this model was according to the ROC curve in order to establish the validity of the gender model:

```
# Set workflow
final_fit_gen <-
  final_wf_gen %>%
  last_fit(ms_split_gen)

# train
final_fit_gen %>%
  collect_metrics()
```

```
## # A tibble: 2 x 4
##   .metric  .estimator .estimate .config
##   <chr>    <chr>          <dbl> <chr>
## 1 accuracy binary         0.845 Preprocessor1_Model1
## 2 roc_auc  binary         0.898 Preprocessor1_Model1
```

```
# collect metrics
final_fit_gen %>%
  collect_predictions() %>%
  roc_curve(gender, .pred_F) %>%
  autoplot()
```




```
args(decision_tree)
```

```
## function (mode = "unknown", engine = "rpart", cost_complexity = NULL,
##     tree_depth = NULL, min_n = NULL)
## NULL
```

In contrast to the status classification model, the ROC curves show that in this model the sensitivity was higher than the specificity. Meaning, the gender classifier model performed much better than the status one. However, the differences between the results of the models of the most important features in the classification of the models raises the question which is the most important in MS. If we the race factor wasn't skewed towards white ethnicity, we could have compared it to its classification model. But in these cases the random forest isn't as effective as always, what leads us to refer to the more reliable model of the two, the gender model. To sum up this part, the classification model depending on the status of health didn't show in the overall analysis any reliability in good classification. Hence, there is great significance to its results. In conclusion, despite the inconsistency in the results of the random forest models, at least the gender model is more robust and useful for classification that is independent from our data set. The gender classification model was better than the health status one, as the ROC curve showed. What proves once more that the random forest is one of the strongest and robust classification algorithms in most cases. In both factor the

initial model before tuning and cross reference had the best accuracy. What leads us to suggest that using the initial model is the best one to use for future classification of similar data. Let's try another classification model in order to ascertain whether there is consistency with the results the gender model showed.

T-distributed Stochastic Neighbor Embedding (t-SNE)

In the random forest model, one of the more robust and stringent classification model, the results still require improvement, at least in some factors. A dimensional reduction could help in raising the classification accuracy by modeling similar objects as nearby points and dissimilar objects as distant points. t-SNE, as a nonlinear dimension reduction technique, can be well-suited for our data since it can visualize a large amount of data with well-separated clusters. For this purpose, we'll have to arrange our data a bit differently:

```
# Numbering our rows and removing the record id in a different variable than the original data
ms_dat_mut <- ms_dat

ms_dat_mut <- ms_dat_mut %>%
  select(-record_id)%>%
  mutate(ID=row_number())

# Selecting the classifying factors
ms_meta <- ms_dat_mut %>%
  select(ID, status, gender, race)
```

In addition, t-SNE requires the selection of the numerical columns as well as standardization of the data, and only then we can perform t-SNE:

```
set.seed(142)
tSNE_ms_fit <- ms_dat_mut %>%
  select(where(is.numeric)) %>%
  column_to_rownames("ID") %>%
  scale() %>%
  Rtsne()
```

After performing t-SNE, we'll save the two t-SNE components of interest in a data frame in order to combine it with the our classifying factors:

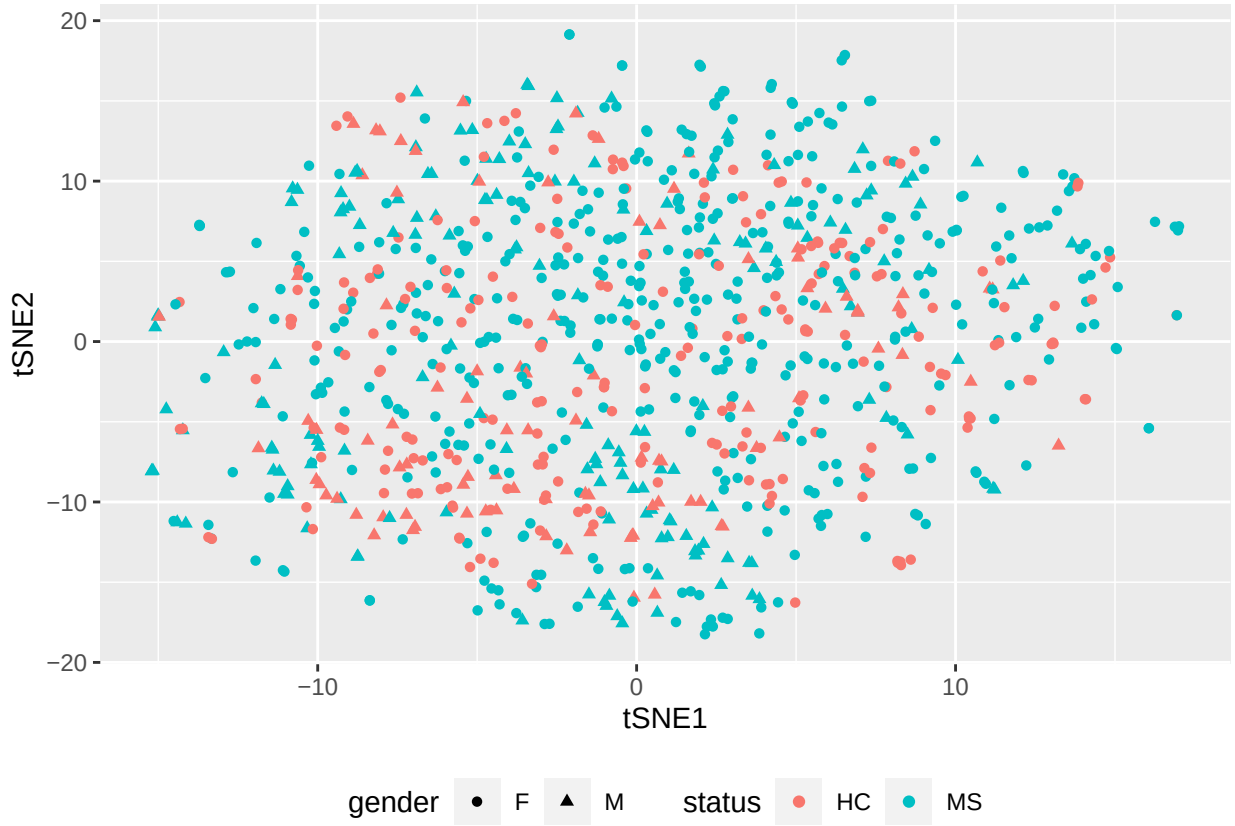
```
# Store 2 t-SNE components in data frame
tSNE_ms_df <- tSNE_ms_fit$Y %>%
  as.data.frame() %>%
  rename(tSNE1="V1",
         tSNE2="V2") %>%
  mutate(ID=row_number())

# Cross reference it to our classifying factors using row number
tSNE_ms_df <- tSNE_ms_df %>%
  inner_join(ms_meta, by="ID")
```

Now let's plot the results above:

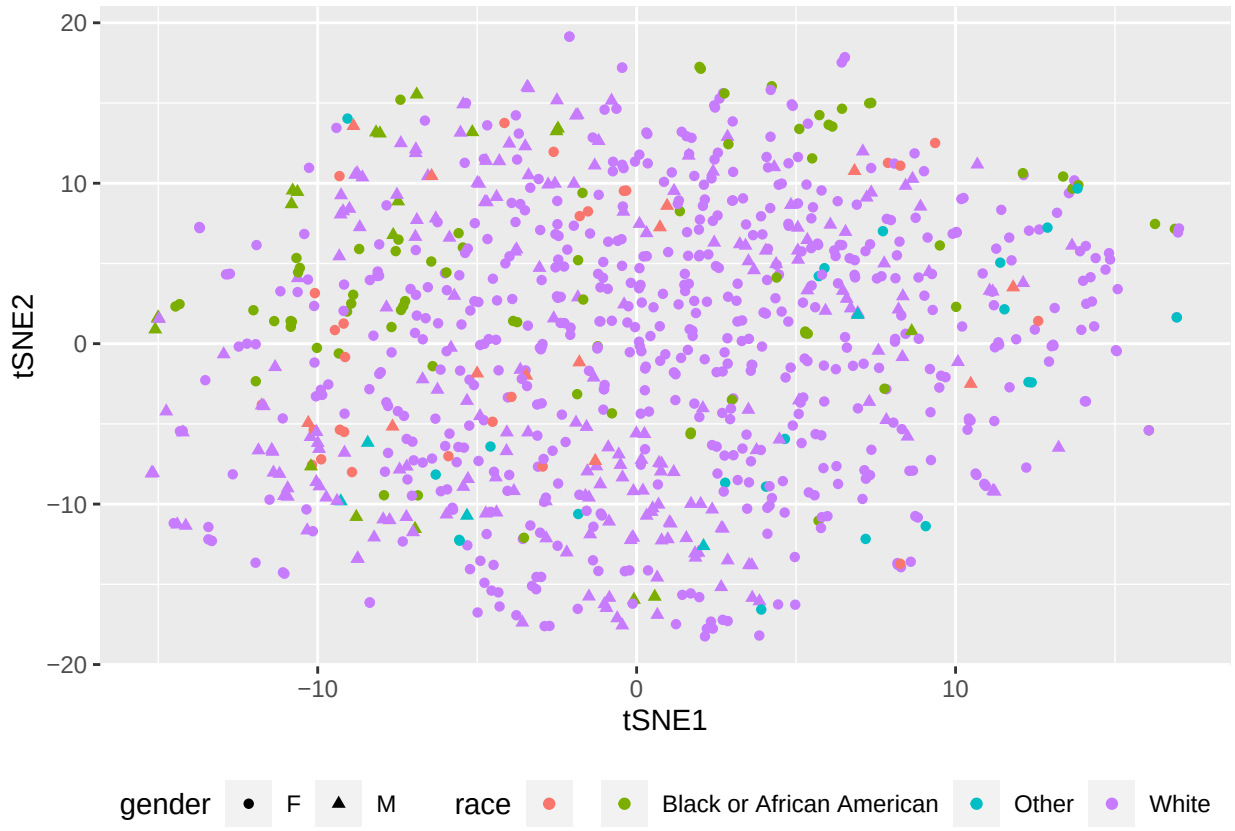
```
tSNE_ms_df %>%
  ggplot(aes(x = tSNE1,
             y = tSNE2,
```

```
            color = status,
            shape = gender))+
  geom_point()+
  theme(legend.position="bottom")
```

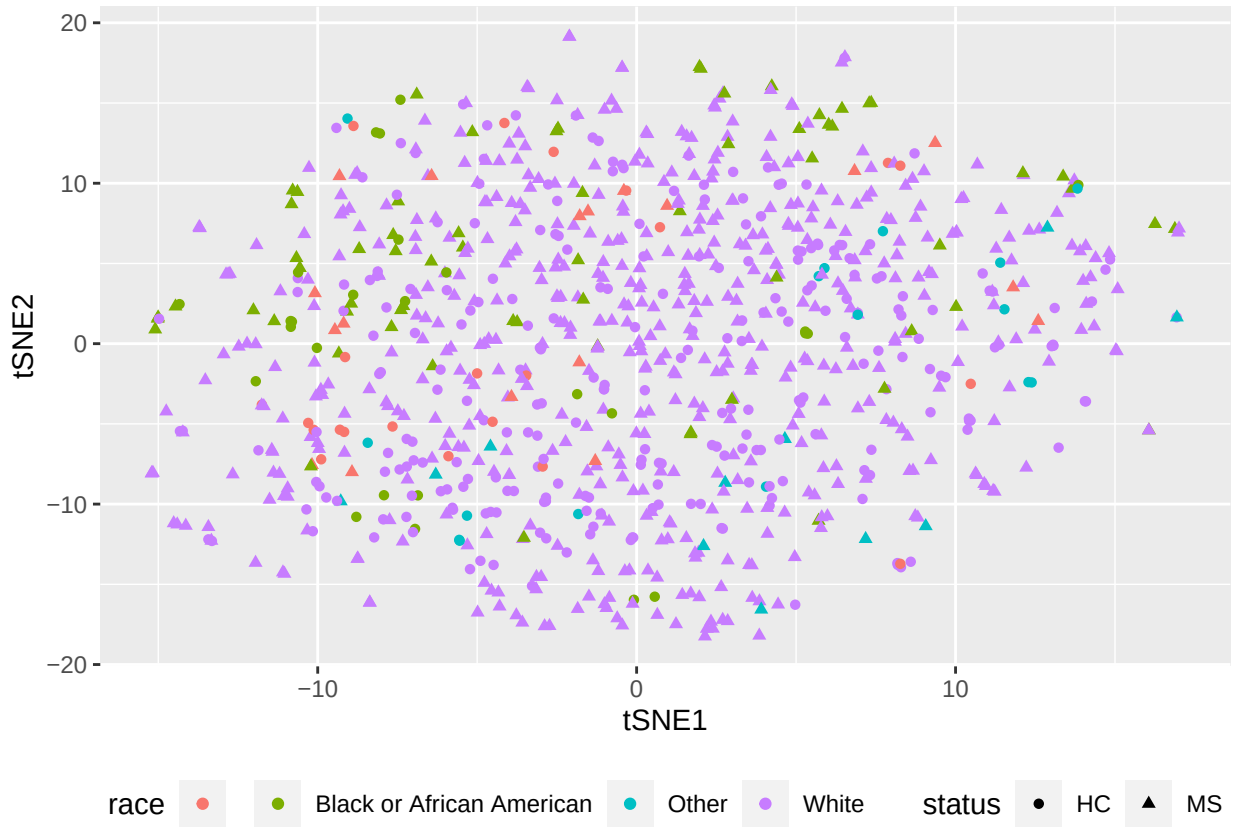


It is apparent that some insights can be taken from the graph above. However, it does coincide with the results of previous section. Status was less efficient in classification than by gender, what can be observed in the great overlap between the factors in the graph. What happens when we add the race factor to the mix?

```
tSNE_ms_df %>%
  ggplot(aes(x = tSNE1,
             y = tSNE2,
             color = race,
             shape = gender))+
  geom_point()+
  theme(legend.position="bottom")
```




```
tSNE_ms_df %>%
  ggplot(aes(x = tSNE1,
             y = tSNE2,
             color = race,
             shape = status))+
  geom_point()+
  theme(legend.position="bottom")
```



These results still cannot help us differentiate between the different factors. It can be a result of the many metabolites that were tested and what we've observed in the classification in previous sections that there was no one metabolite that stood-out in its effect. Moreover, the graph strengthen our previous conclusions that the research has a clear bias toward white ethnicity. The reduction showed as seen in the kmeans section, that the data is continuous and overlapping. Thus, we need to search for other algorithms for classification.

Support Vector Machine (SVM)

In the previous classification sections we assumed that the data was discontinued, but what if it not so as the previous section indicated. An analysis that can help in these cases is the SVM classification algorithm, a supervised classification algorithm, where each data item is plotted as a point in n-dimensional number of features space with the value of each feature being the value of a coordinate. This is followed by finding the hyper-plane that best differentiates the two classes. By SVM's ability to perform a non-linear classification, we can understand the pattern of our data better. First step in SVM is randomizing the numeric data for creating the training and test sets:

```
# Numerical data only
ms_num_fac <- ms_dat %>% select(-record_id, -batch)

# Randomizing the data frame by rows
shuffled_ms <- ms_num_fac[sample(1:nrow(ms_num_fac)), ]
```

Now that the data is randomized, we can create our training and test sets:

```
# Splitting the randomized data into train and test sets
ms_svm_train <- shuffled_ms[1:477, ]
ms_svm_test <- shuffled_ms[478:954, ]
```

Now can build the model, first for the status of health:

```
# Creating and training the model

ms_svm_classifier_stat <- ksvm(ms_svm_train$status ~ ., data = ms_svm_train, kernel = "vanilladot")
```

```
##  Setting default kernel parameters
```

```
ms_svm_classifier_stat
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 1
##
## Linear (vanilla) kernel function.
##
## Number of Support Vectors : 198
##
## Objective Function Value : -10.8347
## Training error : 0
```

The model used 199 support vectors with a training error of 0(?). Is it the same in the test set?

```
# apply model on test set
ms_svm_predictions_stat <- predict(ms_svm_classifier_stat, ms_svm_test)
head(ms_svm_predictions_stat)
```

```
## [1] MS HC MS HC MS MS
## Levels: HC MS
```

To examine how well the MS classifier performed, let's compare the prediction of the health status to what really happens in the test set using the table() function:

```
table(ms_svm_predictions_stat, ms_svm_test$status)
```

```
##
## ms_svm_predictions_stat  HC  MS
##                      HC  76  82
##                      MS  87 232
```

But this table is still quite wage. We can use a binary vector to calculate the percentage to get a clearer picture:

```r
# Calculate true/false values
agreement <- ms_svm_predictions_stat == ms_svm_test$status
# True/false performance percentage
prop.table(table(agreement))
```

```
## agreement
##     FALSE      TRUE
## 0.3542977 0.6457023
```

Meaning, our current SVM status model has a moderate ~71% success rate. Improving the model performance is necessary. A standard Gaussian Radial basis function (RBF) may help in that:

```r
# Train svm model with RBF kernel function
ms_svm_classifier_rbf_stat <- ksvm(ms_svm_train$status ~ ., data = ms_svm_train, kernel = "rbfdot")
```

We'll create the prediction with this kernel:

```r
# Predict on test set
ms_svm_predictions_rbf_stat <- predict(ms_svm_classifier_rbf_stat, ms_svm_test)
```

In comparison to the predictions with the previous kernel:

```r
# calculate preformance
agreement_rbf_stat <- ms_svm_predictions_rbf_stat == ms_svm_test$status
prop.table(table(agreement_rbf_stat))
```

```
## agreement_rbf_stat
##     FALSE      TRUE
## 0.2641509 0.7358491
```

We can see no significant improvement as we saw in previous sections of classifier model of status of health. Maybe classifying by gender will show better results:

```r
# Creating and training the model

ms_svm_classifier_gen <- ksvm(ms_svm_train$gender ~ ., data = ms_svm_train, kernel = "vanilladot")
```

```
##  Setting default kernel parameters
```

```r
ms_svm_classifier_gen
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 1
##
## Linear (vanilla) kernel function.
##
## Number of Support Vectors : 176
##
## Objective Function Value : -4.5452
## Training error : 0
```

The model used 178 support vectors with a training error of 0(?). The predictions in the test set is:

```
# apply model on test set
ms_svm_predictions_gen <- predict(ms_svm_classifier_gen, ms_svm_test)
head(ms_svm_predictions_gen)
```

```
## [1] M F F F M M
## Levels: F M
```

This MS classifier performed in comparison to the test set:

```
table(ms_svm_predictions_gen, ms_svm_test$gender)
```

```
##
## ms_svm_predictions_gen   F   M
##                      F 293  56
##                      M  49  79
```

Or in percentages:

```
# Calculate true/false values
agreement <- ms_svm_predictions_gen == ms_svm_test$gender
# True/false performance percentage
prop.table(table(agreement))
```

```
## agreement
##     FALSE      TRUE
## 0.2201258 0.7798742
```

The gender SVM status model yields a moderate ~75% success rate. Maybe a standard Gaussian Radial basis function (RBF) may help with that:

```
# Train svm model with RBF kernel function
ms_svm_classifier_rbf_gen <- ksvm(ms_svm_train$gender ~ ., data = ms_svm_train, kernel = "rbfdot")
```

With the prediction of:

```
# Predict on test set
ms_svm_predictions_rbf_gen <- predict(ms_svm_classifier_rbf_gen, ms_svm_test)
```

In comparison to the predictions with the previous kernel:

```
# calculate preformance
agreement_rbf <- ms_svm_predictions_rbf_gen == ms_svm_test$gender
prop.table(table(agreement_rbf))
```

```
## agreement_rbf
##     FALSE      TRUE
## 0.1446541 0.8553459
```

In contrast to the previous SVM model, the classification of gender improved significantly with the rbf kernel, from 75% to 87%. Will SVM succeed where previous algorithms failed?

```
# Creating and training the model
ms_svm_classifier_race <- ksvm(ms_svm_train$race ~ ., data = ms_svm_train, kernel = "vanilladot")
```

```
##  Setting default kernel parameters
```

```
ms_svm_classifier_race
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 1
##
## Linear (vanilla) kernel function.
##
## Number of Support Vectors : 247
##
## Objective Function Value : -0.0834 -0.0495 -0.3442 -0.0637 -0.3227 -0.3349
## Training error : 0
```

The model used 260 support vectors with a training error of 0(?) with the predictions of:

```
# apply model on test set
ms_svm_predictions_race <- predict(ms_svm_classifier_race, ms_svm_test)
head(ms_svm_predictions_race)
```

```
## [1] White                       White
## [3] White                       White
## [5] Black or African American
## Levels:  Black or African American Other White
```

It seemed the classification of race worked in this algorithm. Let's check it:

```
table(ms_svm_predictions_race, ms_svm_test$race)
```

```
##
## ms_svm_predictions_race         Black or African American Other White
##                              3                          1     0    13
##   Black or African American  2                         32     1     7
##   Other                      0                          1     1     8
##   White                     19                         21    10   358
```

Or in percentages:

```
# Calculate true/false values
agreement <- ms_svm_predictions_race == ms_svm_test$race
# True/false performance percentage
prop.table(table(agreement))
```

```
## agreement
##     FALSE      TRUE
## 0.1740042 0.8259958
```

The SVM race classification model was the first to overcome the bias in the data and predict with ~86% accuracy the race of MS vs. HC individuals. Maybe radial classification will perfect the accuracy (not over fitting!) due to the previous graph we observed in the previous section (??????????????).

```
# Train svm model with RBF kernel function
ms_svm_classifier_rbf_race <- ksvm(ms_svm_train$race ~ ., data = ms_svm_train, kernel = "rbfdot")
```

With the prediction of test set of:

```
# Predict on test set
ms_svm_predictions_rbf_race <- predict(ms_svm_classifier_rbf_race, ms_svm_test)
```

In comparison to the predictions with the previous kernel:

```
# Calculate performance
agreement_rbf <- ms_svm_predictions_rbf_race == ms_svm_test$race
prop.table(table(agreement_rbf))
```

```
## agreement_rbf
##     FALSE      TRUE
## 0.1907757 0.8092243
```

The classification of race didn't change when we changed the kernel (85-86%). To sum up this section, the SVM algorithm is more flexible than previous ones. It gave overall great predictions, and can overcome biases in the data set. However, the SVM is quite of a black box of an algorithm. The question is if other flexible algorithms can yield similar results and performances.

Conclusions

In this assignment we've use 4 different classifier models: KNN, kmeans, random forest and SVM; and one dimension reduction algorithm: t-SNE on the data set of Fitzgerald et al. 2021 on the MS metabolom. We've tried to classify the data according to three main factors characterizing of the each of the individuals tested in the research: status of health, gender and race. The first, the health status factor, in many of the algorithms didn't give a good classification. The best classification was in the random forest algorithm with 73% accuracy, that even that isn't a good result result. This could be explained by the rational that classifying MS from a healthy population is still enough for a good classification even with the information of the metabolom. It can be assumed that the disease is complex enough from a healthy individual that the metabolom information is still not enough in order to distinguish a healthy individual from MS one. In contrast, in most algorithms the gender factor had the best classifier models of the three. The best one and with the most robust algorithm was random forest of 91% accuracy. This can indicate that in contrast to identifying MS from the general population as observed in the status factor, identifying MS from a female population with the metabolom gives a big accuracy of detection. Even more so, one of the contributing factors for classification was the age which even can further assist doctors in diagnosing MS more easily. Finally, the race factor in most algorithms wasn't classified because of how skewed it is toward the white ethnicity. Most algorithms couldn't overpass the fact that out of four levels of race, 83% were of white ethnicity. Only SVM was able to surpass that and give an unbiased good classification of ~86%. For future experiments, if possible, the testing needs to be more balanced. In the most affecting metabolite area, the results were not conclusive. As Fitzgerald et al 2021 showed, we do see how the aromatic amino acids impact the classification of MS in the status of health as we see aromatic amino acids precursors there. However, from the basis of this report only, much more research is required to get a clearer picture on the connection of the metabolom to MS detection. For example, the best and the most valid classification model we had, random forest by gender, indicated on other amino acids and hormones not specifically associated with aromatic

amino acids. In a broader perspective each algorithm had its own benefits and shortcomings. The first, the KNN model, was simplest to perform. However, its accuracy, sensitivity, specificity and the ability to predict false values (negative and positive) were the worst of the 4 algorithms. It was good for the start of the analysis, to get some sense what the data is, but it is good for this purpose only. To strengthen its results, at least one additional model is required to validate its findings. The second, Kmeans also was easy to perform and illustrated the data wonderfully. But at the same time, its accuracy and general estimates were quite wage. It gave three distinct groups in the complex data, but from the many numbers there was only a wage way to check the validity of the results. Like KNN, this can also be a starter for getting to know our data, but its results need to be complimented with at least one other classification model. The third, random forest, was the most robust and stringent algorithms of the 4, maybe due to its categorical classification that is not dependent on the scatter of the data. Beyond the good classification it gave to the status of health and gender, it has enough methods of correction and improvement that other algorithms didn't have. Moreover, through the different processes the model had estimates to understand the model's performance, which weren't in the other algorithms. However, because it's one of the most robust and stringent algorithm, it is sensitive to imbalances in the data. In our case the race factor is biased towards the white ethnicity, which resulted with the analysis not working on this factor. With that said, this can be a characteristic of many algorithms and not just random forest. In contrast to the random forest, the final algorithm, the SVM algorithm is much more flexible. It fits continuous data with an ability to adjust to the correlation within the data, such as linear correlation as well as radial. Hence, it is less sensitive to imbalanced data as the factor race is and was able to give a good classification. Even its accuracy was quite good in comparison to the other algorithms excluding the random forest. But still it didn't give the best classification outcome of all algorithms. Even more so, the algorithm is quite of a black box without any estimates to review and check the validity of the model in the process. The dimension reduction method we've used was t-SNE in order to have an additional to the PCA used in the paper. The t-SNE indded reduced much of the clutter observed in the kmeans section. Despite that the results couldn't help in differentiation between the different factors. It can be a result of the many metabolites that were tested, or that the method used in Fitzgerald et al. 2021 PCA was more suited to the data than t-SNE. Otherwise, if the result of the method is used afterwards in an additional classification model, it can yield better classification models. In conclusion, further research is required to comprehend the differences between this report and the extensive research of Fitzgerald et al. 2021.

Refrences

Fitzgerald KC, Smith MD, Kim S, Sotirchos ES, Kornberg MD, Douglas M, Nourbakhsh B, Graves J, Rattan R, Poisson L, Cerghet M, Mowry EM, Waubant E, Giri S, Calabresi PA, Bhargava P. Multi-omic evaluation of metabolic alterations in multiple sclerosis identifies shifts in aromatic amino acid metabolism. Cell Rep Med. 2021 Oct 19;2(10):100424. doi: 10.1016/j.xcrm.2021.100424. PMID: 34755135; PMCID: PMC8561319.

Levi I, Gurevich M, Perlman G, Magalashvili D, Menascu S, Bar N, Godneva A, Zahavi L, Chermon D, Kosower N, Wolf BC, Malka G, Lotan-Pompan M, Weinberger A, Yirmiya E, Rothschild D, Leviatan S, Tsur A, Didkin M, Dreyer S, Eizikovitz H, Titngi Y, Mayost S, Sonis P, Dolev M, Stern Y, Achiron A, Segal E. Potential role of indolelactate and butyrate in multiple sclerosis revealed by integrated microbiome-metabolome analysis. Cell Rep Med. 2021 Apr 20;2(4):100246. doi: 10.1016/j.xcrm.2021.100246. PMID: 33948576; PMCID: PMC8080254.